

WINDOWS EDR THREAT DETECTION & INCIDENT RESPONSE USING LIMACHARLIE

Design and Implementation of a Mini SOC Environment for Windows Endpoint Monitoring, Detection Engineering, Threat Investigation, MITRE ATT&CK Mapping, and Incident Response

A Cybersecurity / SOC Analyst Laboratory Project Report

Prepared by: Ranbir Mishra

Institution / Organization: Not specified

Report date: 2 October 2026

Certificate / Declaration

I declare that this report documents a private, controlled cybersecurity laboratory using benign, non-destructive activity. Statements about detections, alerts, and incident outcomes are limited to what is established by the evidence included in this report.

Student/Analyst Name: Ranbir Mishra

Signature: Not provided

Date: 2 October 2026

Acknowledgement

I would like to thank the authors and maintainers of LimaCharlie, MITRE ATT&CK, Microsoft Learn, and NIST for the technical documentation cited in this report. These resources informed the lab design and the discussion of endpoint telemetry, detection engineering, and incident response. Any remaining errors or interpretations are my responsibility.

Abstract

This report documents a small Windows endpoint monitoring lab using LimaCharlie. The supplied screenshots establish a Windows 10 Pro endpoint baseline, show a LimaCharlie sensor online, and record benign PowerShell, connectivity-check, and scheduled-task activity. They also show a scheduled task being created and later removed. Although detection rules are visible in the console, the supplied evidence does not show a custom rule firing or an alert being investigated; the D&R rules page reports 0 of 13 rules tested. Accordingly, this report records verified facts, distinguishes planned scenarios from observed results, and marks unsupported incident details as not captured.

Table of Contents

Chapter 1 — Introduction.....	10
Chapter 2 — Literature and Technologies.....	12
Chapter 3 — Lab Environment and System Design	13
Chapter 4 — Screenshot and Evidence Standard	14
Module 1 — Windows Endpoint Setup	16
Module 2 — LimaCharlie EDR Deployment.....	26
Module 3 — Endpoint Telemetry Verification	33
Module 4 — Detection Engineering	37
Module 5 — Incident 1: Suspicious PowerShell	45
Module 6 — Incident 2: Suspicious Process Chain	54
Module 7 — Incident 3: Persistence Detection.....	60
Module 8 — Incident 4: Suspicious Network Activity	65
Module 9 — Threat Investigation Workflow & Evidence Collection.....	70
Module 10 — MITRE ATT&CK Mapping	72
Module 11 — Incident Response.....	73
False Positives and Detection Tuning	75
Module 12 — SOC Dashboard and Final Results	76
Screenshot Master List	77
General Troubleshooting Reference	84
Chapter 5 — Controlled Attack/Activity Simulation (Summary)	85
Chapter 6 — Detection and Investigation (Summary)	85
Chapter 7 — Incident Response (Summary)	85
Chapter 8 — Results	85
Skills Demonstrated	87
Module 13 — Final Incident Report Summary	88
Chapter 9 — Conclusion	89
References	90
Appendix	91

List of Figures

Figure 1.1 — Windows VM Powered On and Logged In	16
Figure 1.2 — Windows Endpoint Hostname	17
Figure 1.3 — Logged-In User Identity	18
Figure 1.4 — Windows Version and Architecture	19
Figure 1.5 — CPU and RAM Configuration	19
Figure 1.6 — Disk / Storage Configuration	20
Figure 1.7 — IP Configuration	20
Figure 1.8 — Internet Connectivity Test	22
Figure 1.9 — PowerShell Version	22
Figure 1.10 — Windows Event Viewer Baseline	23
Figure 1.11 — Windows Defender / Security Status	24
Figure 1.12 — Administrator Privilege Check	24
Figure 2.1 — LimaCharlie Account / Login Screen	27
Figure 2.2 — LimaCharlie Organization Created/Selected	27
Figure 2.3 — Sensor Installer / Installation Key Generated	28
Figure 2.4 — Sensor Installation in Progress / Complete	30
Figure 2.5 — LimaCharlie Sensor Service Status	30
Figure 2.6 — Endpoint Visible and Online in LimaCharlie Console	31
Figure 3.1 — LimaCharlie Timeline / Event Stream View	33
Figure 3.2 — Process Telemetry Detail	34
Figure 3.3 — Network Telemetry Detail	35
Figure DE.1 — Detection 1 Rule Configuration	39
Figure DE.2 — Detection 2 Rule Configuration	41
Figure DE.3 — Detection 3 Rule Configuration	42
Figure DE.4 — Detection 4 Rule Configuration	43
Figure INC1.1 — PowerShell Test Activity Executed	46
Figure INC1.2 — Detection Rule Triggered	47
Figure INC1.3 — Alert Generated	47
Figure INC1.4 — Process Detail	48
Figure INC1.5 — Command-Line Detail	49
Figure INC1.6 — Investigation View	49
Figure INC1.7 — MITRE ATT&CK Mapping	50
Figure INC1.8 — Response Action	51

Figure INC1.9 — Verification	51
Figure INC2.1 — Process Chain Generated	55
Figure INC2.2 — Detection Rule Triggered (cmd->PowerShell)	55
Figure INC2.3 — Full Process Tree in LimaCharlie.....	56
Figure INC2.4 — Command Lines for Each Process in the Chain	56
Figure INC2.5 — Investigation Summary	57
Figure INC3.1 — Scheduled Task Creation Command	61
Figure INC3.2 — Detection Rule Triggered (Scheduled Task).....	61
Figure INC3.3 — Alert and Task Detail in LimaCharlie	62
Figure INC3.4 — Task Scheduler Confirmation	62
Figure INC3.5 — Task Removal (Cleanup)	62
Figure INC3.6 — Post-Cleanup Verification.....	62
Figure INC4.1 — Network Test Activity Executed	66
Figure INC4.2 — Detection Rule Triggered (Process-to-Network)	66
Figure INC4.3 — Network Telemetry Detail.....	67
Figure INC4.4 — Process-to-Network Correlation View	68

List of Tables

Table 1. Hardware and VM Requirements	14
Table 2. Software Components	14
Table 3. Component Purpose and Evidence	15
Table 4. Endpoint Baseline	26
Table 5. Endpoint Setup Troubleshooting	26
Table 6. Sensor Deployment Troubleshooting	33
Table 7. Telemetry Evidence.....	37
Table 8. Missing Telemetry Troubleshooting	38
Table 9. Severity Model	38
Table 10. Incident 1 Evidence	53
Table 11. Incident 1 Timeline.....	53
Table 12. Incident 2 Evidence	59
Table 13. Incident 2 Timeline.....	59
Table 14. Incident 3 Evidence	64
Table 15. Incident 3 Timeline.....	64
Table 16. Incident 4 Evidence	70
Table 17. Incident 4 Timeline.....	70
Table 18. MITRE ATT&CK Mapping.....	73
Table 19. Dashboard Results from Supplied Evidence	77
Table 20. Endpoint Setup Screenshot Index.....	79
Table 21. LimaCharlie Deployment Screenshot Index.....	80
Table 22. Telemetry Screenshot Index	80
Table 23. Detection Engineering Screenshot Index.....	81
Table 24. Incident 1 Screenshot Index.....	81
Table 25. Incident 2 Screenshot Index.....	82
Table 26. Incident 3 Screenshot Index.....	82
Table 27. Incident 4 Screenshot Index.....	82
Table 28. MITRE ATT&CK Screenshot Index	83
Table 29. Incident Response Screenshot Index	83
Table 30. Dashboard Screenshot Index	83
Table 31. Detection Troubleshooting	85
Table 32. Process Tree Troubleshooting	85
Table 33. Detection and Incident Results	86

Table 34. Final Incident Summary89

Chapter 1 — Introduction

1.1 Background

Modern cybersecurity operations depend on the ability to see what is actually happening on an endpoint — not just whether a file matches a known malware signature. Endpoint Detection and Response (EDR) platforms address this by continuously collecting endpoint telemetry (process creation, command-line arguments, network connections, file activity, and user context) and applying detection logic against that telemetry in near real time. This project builds a small, self-contained SOC environment around one Windows endpoint and the LimaCharlie EDR platform in order to practice — end to end — the workflow a junior SOC analyst or detection engineer performs every day.

1.2 Endpoint Security and EDR

Endpoint security has evolved from static, signature-based antivirus toward behavior-based detection and response. Traditional antivirus primarily answers the question “is this file known-bad?” EDR instead answers “what is this process doing, and is that behavior suspicious in context?” by recording rich telemetry and evaluating it against detection rules, then giving an analyst the tools to investigate, contain, and remediate. This project treats LimaCharlie as that EDR layer, sitting on top of the native Windows telemetry sources (process creation, PowerShell activity, network connections, and scheduled tasks).

1.3 Security Operations Center (SOC) Context

A SOC is the organizational people, process, and technology function responsible for monitoring, detecting, and responding to security events. This project mimics — at a single-endpoint, single-analyst scale — the core SOC functions: telemetry ingestion, detection engineering, alert triage, investigation, threat-intelligence mapping (MITRE ATT&CK), incident response, and reporting.

1.4 Problem Statement

The project is designed to address the visibility gap in a controlled lab by instrumenting a Windows VM with LimaCharlie and documenting detections for four representative behavior categories. The supplied screenshots establish endpoint setup and limited benign activity; they do not demonstrate a complete detect-investigate-respond-report cycle for all four scenarios.

1.5 Objectives

- Monitor a Windows VM using an EDR sensor.
- Collect endpoint telemetry (process, network, user, file, and event data) using LimaCharlie.
- Detect suspicious activities through custom detection rules.
- Investigate alerts using process trees, command lines, users, and network context.
- Map observed behavior to MITRE ATT&CK techniques.
- Perform controlled, safe incident-response actions.
- Generate a professional, evidence-driven incident report.

1.6 Scope

The project scope is limited to a single Windows 10/11 virtual machine, a single LimaCharlie sensor/organization, four controlled and non-destructive incident scenarios (PowerShell execution, process-chain anomaly, benign scheduled-task persistence, and controlled outbound network activity), custom detection rules authored for those four scenarios, and the investigation, response, and reporting workflow built around them.

1.7 Limitations

- This is a single-endpoint lab, not a multi-host enterprise SOC deployment.
- The planned scenarios use benign, self-generated activity; the supplied evidence does not establish that every scenario was executed or that any real adversary or malware was involved.
- LimaCharlie's exact UI labels, rule syntax, and feature names may change over time and must be verified directly against the current LimaCharlie console and documentation rather than assumed from this report.
- Results and screenshots in this report are limited to evidence personally captured and supplied for review. Where the evidence does not establish an outcome, the report marks it as not captured rather than treating it as a completed test.

Chapter 2 — Literature and Technologies

2.1 Endpoint Detection and Response (EDR)

EDR platforms continuously record endpoint activity (telemetry) and evaluate it against detection logic to surface suspicious behavior for analyst review. Core EDR capabilities used in this project include process telemetry, command-line visibility, network-connection telemetry, and the ability to write and test custom detection rules.

2.2 LimaCharlie

LimaCharlie is a cloud-based, SecOps-focused platform that exposes EDR sensor telemetry and detection/response capability through a console and API. In this project it is used as the sole EDR platform: a lightweight sensor is installed on the Windows VM, telemetry flows to a LimaCharlie organization, and detection rules are authored to alert on the project's four controlled scenarios. Exact menu names, rule syntax, and workflow steps should be verified against the live LimaCharlie console at the time of the lab, since SaaS interfaces evolve.

2.3 Windows Endpoint Telemetry Sources

- Process creation events — process name, PID, parent process, command line, user, and timestamp.
- PowerShell activity — script block and command-line telemetry for interpreter-based execution.
- Scheduled tasks — creation of persistence mechanisms via schtasks / Task Scheduler.
- Network connections — destination IP/host, port, protocol, and the originating process.

2.4 MITRE ATT&CK

MITRE ATT&CK is a publicly maintained knowledge base of adversary tactics and techniques observed in real-world intrusions. Mapping detected lab activity to ATT&CK technique IDs (for example T1059.001 for PowerShell) gives the report a standardized vocabulary for describing behavior and lets the mapping be compared against industry-standard threat models.

2.5 Incident Response Lifecycle

This project follows the widely used incident-response lifecycle: Preparation, Detection, Analysis, Containment, Eradication, Recovery, and Lessons Learned. Each controlled incident in this report is walked through Detection, Analysis, Containment, Eradication, Recovery, and Lessons Learned explicitly.

2.6 Detection Engineering

Detection engineering is the practice of translating a specific behavior of interest (e.g., “cmd.exe spawning powershell.exe”) into a testable, tunable detection rule, then validating that rule against real telemetry, tuning it to reduce false positives, and documenting its logic and limitations.

Chapter 3 — Lab Environment and System Design

3.1 Hardware / VM Requirements

Table 1. Hardware and VM Requirements

Requirement	Minimum Specification	Purpose
Operating System	Windows 10 or Windows 11	Monitored endpoint OS
RAM	4 GB or more	Run OS, sensor, and test workloads smoothly
CPU	2 cores or more	Support concurrent OS + sensor + test activity
Disk	40 GB or more free	OS, logs, sensor, and evidence storage
Network	Internet connectivity	Sensor-to-cloud communication with LimaCharlie
Account	Local Administrator	Install sensor, run PowerShell/schtasks commands

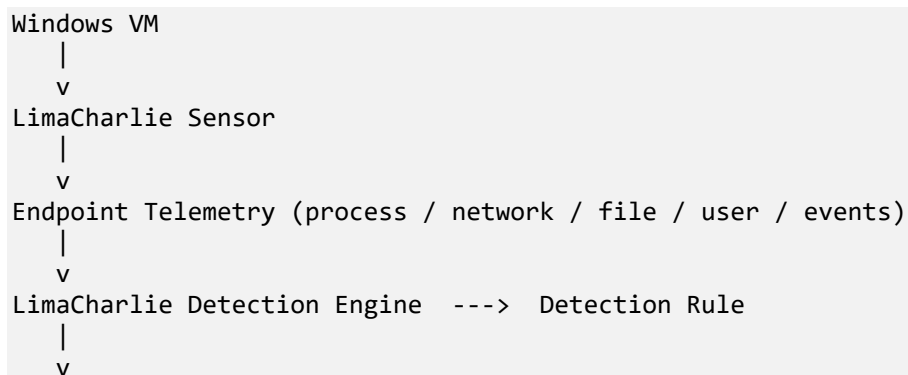
3.2 Software Components

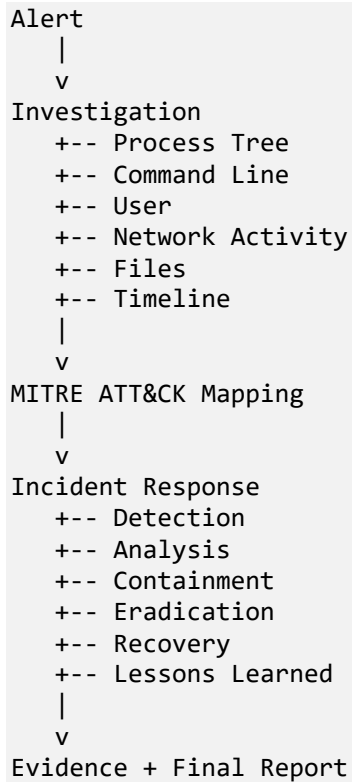
Table 2. Software Components

Component	Why It Is Used
LimaCharlie Sensor	Provides EDR telemetry collection, detection engine, and response actions for the endpoint.
PowerShell	Used both to generate controlled test activity and to query endpoint configuration for baselining.
Windows Event Viewer	Cross-reference native Windows event logs against LimaCharlie telemetry during investigation.
Windows Defender / Windows Security	Confirms native OS security posture and shows how EDR complements built-in AV.
Web Browser	Access the LimaCharlie web console.
Sysmon (optional)	Optional additional telemetry source (e.g., richer process and network logging) to cross-validate LimaCharlie visibility.

3.3 Lab Architecture

The end-to-end data and workflow architecture used throughout this project is:





3.3.1 Component Purpose and Evidence Table

Table 3. Component Purpose and Evidence

Component	Purpose	Evidence
Windows VM	Monitored endpoint	Screenshot
LimaCharlie Sensor	Endpoint telemetry collection	Screenshot
Detection Engine	Detect suspicious behavior	Screenshot
Alert	Security event notification	Screenshot
Investigation	Analyze activity	Screenshot
MITRE ATT&CK	Technique mapping	Table / Screenshot
Incident Response	Controlled response	Screenshot
Report	Documentation	Final evidence

3.4 Data Flow Summary

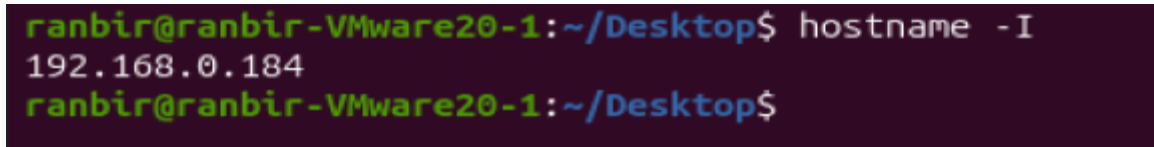
This paragraph describes the intended data flow. The screenshots establish sensor presence, rule configuration, and selected endpoint activity; they do not document a complete alert-to-response sequence.

Chapter 4 — Screenshot and Evidence Standard

Screenshots are included where available. Each observation below describes what the supplied image establishes and identifies evidence that is missing or mismatched; it does not infer a result from a figure title alone.

Example: Figure 1.1 — Windows Endpoint Hostname

Purpose: Verify the hostname of the monitored Windows endpoint.



```
ranbir@ranbir-VMware20-1:~/Desktop$ hostname -I
192.168.0.184
ranbir@ranbir-VMware20-1:~/Desktop$
```

Evidence to capture:

- Hostname
- PowerShell command
- Windows environment / terminal window

Observation: Not captured in supplied evidence for this figure.

This project never simply instructs “take a screenshot.” Every figure states exactly what must be visible, why the evidence matters, and — where relevant — what must NOT be exposed (e.g., real personal IP addresses, unrelated credentials, or other unrelated windows/tabs).

Module 1 — Windows Endpoint Setup

1. Objective

Establish and document the Windows VM as the monitored endpoint, and record a complete configuration baseline before the LimaCharlie sensor is installed.

2. Background

A documented pre-deployment baseline (hostname, user, OS version, hardware, network configuration, and security posture) gives the investigation phase a reference point: any later deviation observed in telemetry can be compared back to this known-good state.

3. Prerequisites

- A Windows 10/11 VM meeting the hardware requirements in Section 3.1.
- Administrator access to the VM.
- Internet connectivity for the VM.

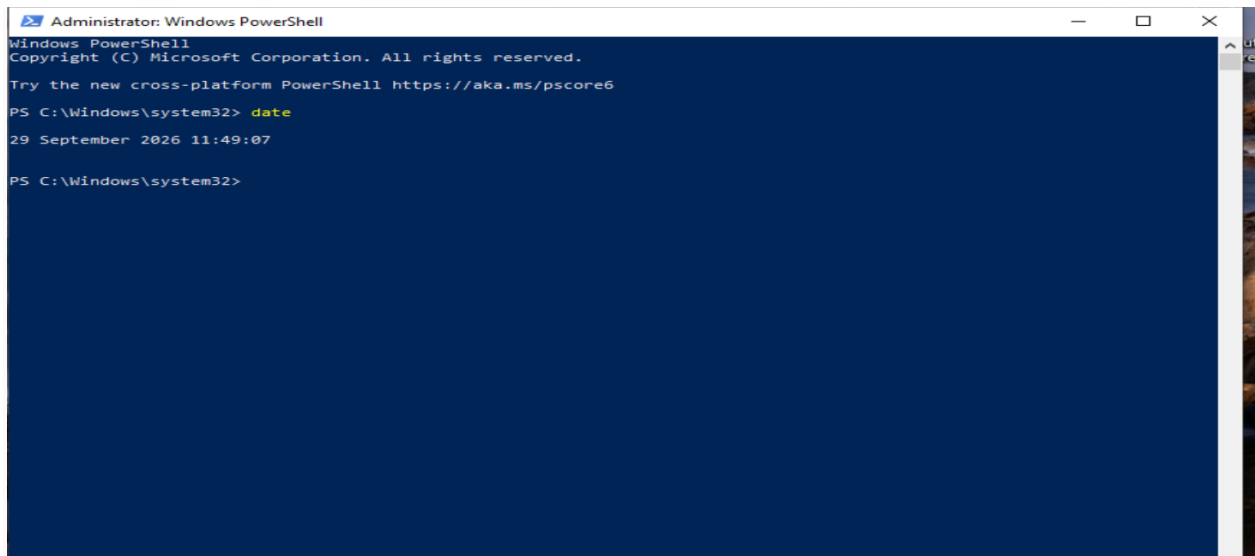
4. Step-by-Step Procedure

1.1 Start and Verify the Windows VM

Power on the Windows VM from your hypervisor (e.g., VirtualBox, VMware, Hyper-V, or a cloud VM console). Confirm the VM boots to the Windows desktop and that you can log in with the administrator account.

Figure 1.1 — Windows VM Powered On and Logged In

Purpose: Confirm the monitored endpoint is running and accessible before any telemetry collection begins.



Evidence to capture:

- Windows desktop
- VM window/console title bar (hypervisor name)
- Taskbar clock/date

Observation: Not captured in supplied evidence for this figure.

1.2 Verify Hostname

Command:

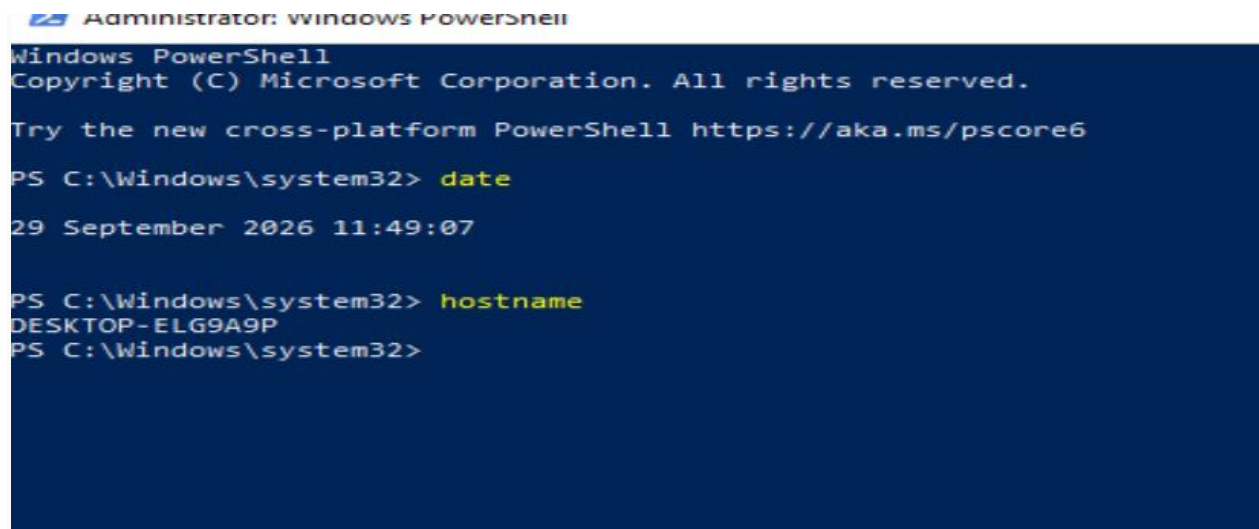
```
hostname
```

Returns the NetBIOS computer name of the endpoint. The baseline value recorded here comes from the supplied hostname screenshot; incident-specific telemetry should be used to confirm host identity for each event.

Expected result: A single line printing the machine's configured hostname.

Figure 1.2 — Windows Endpoint Hostname

Purpose: Verify the hostname of the monitored Windows endpoint.



Evidence to capture:

- Hostname value
- PowerShell command
- Prompt/terminal window

Observation: Not captured in supplied evidence for this figure.

1.3 Verify Logged-In User

Commands:

```
whoami
$env:USERNAME
$env:USERDOMAIN
```

Explanation: `whoami` prints the fully qualified DOMAIN\username of the current session. `\$\_env:USERNAME` and `\$\_env:USERDOMAIN` read the same information from environment variables, useful for scripting and for cross-checking against LimaCharlie's reported user field on telemetry events. Expected result: Consistent username/domain values across all three commands.

Figure 1.3 — Logged-In User Identity

Purpose: Verify the identity of the user account operating the monitored endpoint.

```
PS C:\Windows\system32> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::473b:bd3a:eea7:795e
    IPv4 Address. . . . . : 192.168.0.183
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.0.1
PS C:\Windows\system32> whoami
>> $_env:USERNAME
>> $_env:USERDOMAIN
desktop-elg9a9p\ranbir
ranbir
DESKTOP-ELG9A9P
PS C:\Windows\system32>
```

Evidence to capture:

- Output of whoami
- Output of \$_env:USERNAME and \$_env:USERDOMAIN

Do NOT expose in this screenshot:

- Do not expose real personal usernames if this report will be shared publicly — consider using a lab-specific account name.

Observation: Not captured in supplied evidence for this figure.

1.4 Verify Windows Version

```
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion,
OsArchitecture
```

Explanation: Retrieves the Windows edition, build/version string, and CPU architecture (x64/ARM64), all useful context when interpreting telemetry or troubleshooting sensor compatibility.

Expected result: A small table listing the product name, version, and architecture.

Figure 1.4 — Windows Version and Architecture

Purpose: Confirm the exact Windows edition and architecture of the monitored endpoint.

```
PS C:\Windows\system32> whoami
>> $env:USERNAME
>> $env:USERDOMAIN
desktop-elg9a9p\ranbir
ranbir
DESKTOP-ELG9A9P
PS C:\Windows\system32> Get-ComputerInfo | Select-Object WindowsProductName,WindowsVersion,OsArchitecture

WindowsProductName WindowsVersion OsArchitecture
-----
Windows 10 Pro      2009                64-bit

PS C:\Windows\system32> █
```

Evidence to capture:

- WindowsProductName
- WindowsVersion
- OsArchitecture

Observation: Not captured in supplied evidence for this figure.

1.5 Verify CPU / RAM

```
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
NumberOfLogicalProcessors
Get-CimInstance Win32_ComputerSystem | Select-Object TotalPhysicalMemory
```

Explanation: Confirms the CPU model/core count and total physical memory, verifying the VM meets the lab's minimum hardware requirements.

Expected result: CPU name/core counts and a memory size in bytes (divide by 1GB for a human-readable value).

Figure 1.5 — CPU and RAM Configuration

Purpose: Confirm the VM meets minimum hardware requirements.

```
PS C:\Windows\system32> Get-CimInstance Win32_Processor | Select-Object Name,NumberOfCores,NumberOfLogicalProcessors
>> Get-CimInstance Win32_ComputerSystem | Select-Object TotalPhysicalMemory

Name                                     NumberOfCores NumberOfLogicalProcessors
----
13th Gen Intel(R) Core(TM) i5-13420H      2                2
13th Gen Intel(R) Core(TM) i5-13420H      2                2

PS C:\Windows\system32> █
```

Evidence to capture:

- CPU name and core count
- Total physical memory

Observation: Not captured in supplied evidence for this figure.

1.6 Verify Storage

```
Get-Volume | Select-Object DriveLetter, FileSystemLabel, SizeRemaining, Size
```

Explanation: Lists mounted volumes with total and remaining capacity, confirming sufficient free disk space for the OS, sensor, and evidence artifacts.

Figure 1.6 — Disk / Storage Configuration

Purpose: Confirm sufficient free disk space is available on the endpoint.

```
PS C:\Windows\system32> Get-Volume | Select-Object DriveLetter,FileSystemLabel,SizeRemaining,Size

DriveLetter FileSystemLabel SizeRemaining      Size
-----
C           88240128      572518400
D           72525824      100663296
C           32230793216 63726366720
D           ESD-ISO      0 4893306880

PS C:\Windows\system32>
```

Evidence to capture:

- Drive letter
- Total size and remaining space

Observation: Not captured in supplied evidence for this figure.

1.7 Verify IP Configuration

```
ipconfig
Get-NetIPAddress -AddressFamily IPv4
```

Explanation: Confirms the endpoint's assigned IPv4 address(es) and network adapters, which will later be cross-referenced against LimaCharlie's reported network telemetry.

Figure 1.7 — IP Configuration

Purpose: Record the endpoint's network configuration prior to sensor deployment.

```

PS C:\Windows\system32> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::473b:bd3a:eea7:795e%8
    IPv4 Address. . . . . : 192.168.0.183
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.0.1
PS C:\Windows\system32> Get-NetIPAddress -AddressFamily IPv4

IPAddress      : 192.168.0.183
InterfaceIndex  : 8
InterfaceAlias  : Ethernet0
AddressFamily   : IPv4
Type            : Unicast
PrefixLength    : 24
PrefixOrigin    : Dhcp
SuffixOrigin    : Dhcp
AddressState    : Preferred
ValidLifetime   : 01:50:42
PreferredLifetime : 01:50:42
SkipAsSource    : False
PolicyStore     : ActiveStore

IPAddress      : 127.0.0.1
InterfaceIndex  : 1
InterfaceAlias  : Loopback Pseudo-Interface 1
AddressFamily   : IPv4
Type            : Unicast
PrefixLength    : 8
PrefixOrigin    : WellKnown
SuffixOrigin    : WellKnown
AddressState    : Preferred
ValidLifetime   : Infinite ([TimeSpan]::MaxValue)
PreferredLifetime : Infinite ([TimeSpan]::MaxValue)
SkipAsSource    : False
PolicyStore     : ActiveStore

PS C:\Windows\system32> 

```

Evidence to capture:

- Adapter name
- IPv4 address
- Default gateway

Do NOT expose in this screenshot:

- Do not expose your real public IP address if this report will be shared publicly; a private/lab IP range is acceptable to show.

Observation: Not captured in supplied evidence for this figure.

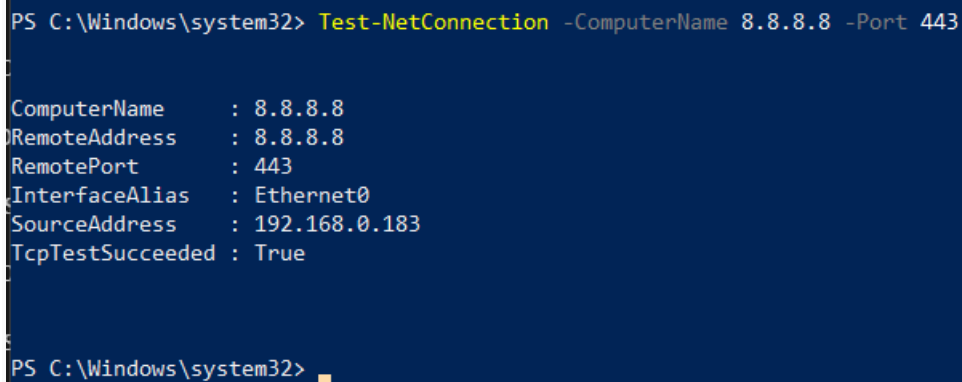
1.8 Verify Internet Connectivity

```
Test-NetConnection -ComputerName 8.8.8.8 -Port 443
```

Explanation: Confirms outbound connectivity is available, which the LimaCharlie sensor requires to reach the cloud platform.

Figure 1.8 — Internet Connectivity Test

Purpose: Confirm outbound connectivity required for the sensor to reach LimaCharlie's cloud service.



```
PS C:\Windows\system32> Test-NetConnection -ComputerName 8.8.8.8 -Port 443

ComputerName      : 8.8.8.8
RemoteAddress     : 8.8.8.8
RemotePort        : 443
InterfaceAlias    : Ethernet0
SourceAddress     : 192.168.0.183
TcpTestSucceeded  : True

PS C:\Windows\system32>
```

Evidence to capture:

- TcpTestSucceeded result
- Target and port tested

Observation: Not captured in supplied evidence for this figure.

1.9 Verify PowerShell Version

```
$PSVersionTable
```

Explanation: Confirms the installed PowerShell version, relevant because later test commands and some detection logic may depend on PowerShell version-specific behavior (e.g., script block logging availability).

Figure 1.9 — PowerShell Version

Purpose: Record the PowerShell version available for generating and investigating test activity.

```
PS C:\Windows\system32> $PSVersionTable

Name                           Value
----                           -
PSVersion                      5.1.19041.6456
PSEdition                      Desktop
PSCompatibleVersions           {1.0, 2.0, 3.0, 4.0...}
BuildVersion                   10.0.19041.6456
CLRVersion                     4.0.30319.42000
WSManStackVersion              3.0
PSRemotingProtocolVersion      2.3
SerializationVersion           1.1.0.1

PS C:\Windows\system32>
```

Evidence to capture:

- PSVersion field
- PSEdition field

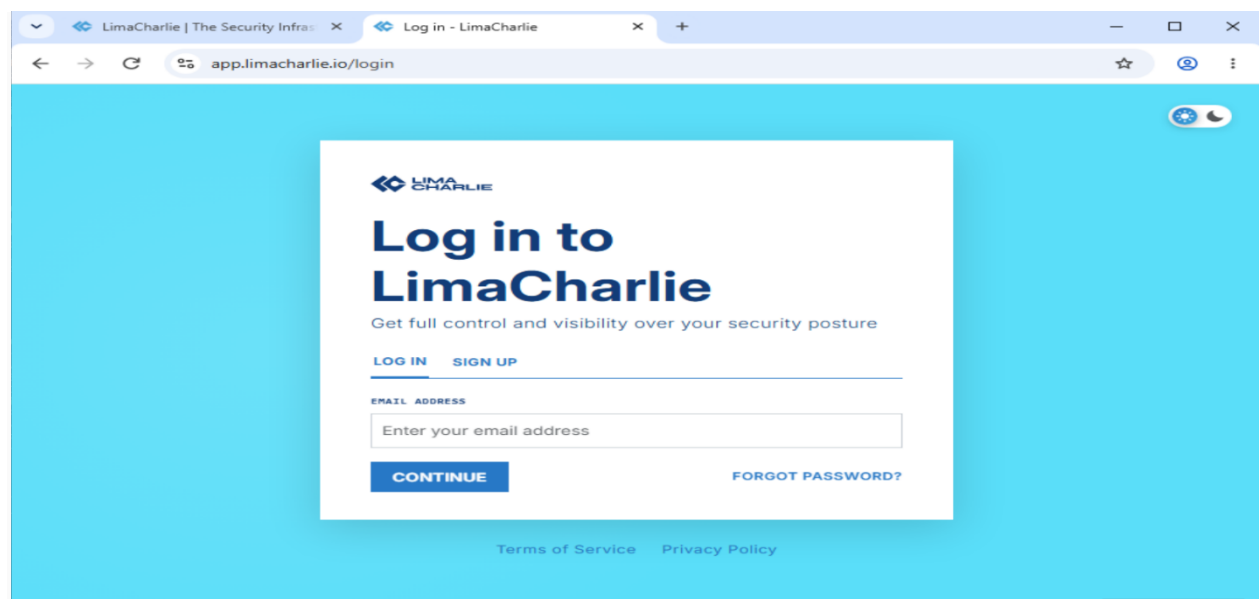
Observation: Not captured in supplied evidence for this figure.

1.10 Verify Windows Event Viewer

Open Event Viewer via Start → “Event Viewer”, or run `eventvwr.msc`. Expand Windows Logs → confirm the Application, Security, and System logs are present and populated.

Figure 1.10 — Windows Event Viewer Baseline

Purpose: Confirm native Windows logging is functioning as a secondary evidence source alongside LimaCharlie telemetry.



Evidence to capture:

- Event Viewer window
- Windows Logs node expanded
- Recent event entries visible

Observation: Not captured in supplied evidence for this figure.

1.11 Verify Windows Defender / Windows Security

Open Windows Security (Start → “Windows Security”) → Virus & threat protection. Confirm real-time protection is enabled and note the current status.

Figure 1.11 — Windows Defender / Security Status

Purpose: Document native antivirus posture prior to sensor deployment, to illustrate how EDR complements built-in AV.



Evidence to capture:

- Real-time protection status
- Virus & threat protection summary

Observation: Not captured in supplied evidence for this figure.

1.12 Verify Administrator Privileges

```
([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
```

Explanation: Returns True/False depending on whether the current PowerShell session has local Administrator rights — required for sensor installation and for the scheduled-task and process-chain test activity in later modules.

Figure 1.12 — Administrator Privilege Check

Purpose: Confirm the session used for lab activity has the administrator rights required for later modules.

```
PS C:\Windows\system32> ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
True
PS C:\Windows\system32> .
```

Evidence to capture:

- Boolean result (True/False)
- Command used

Observation: Not captured in supplied evidence for this figure.

1.13 Establish Endpoint Baseline

Consolidate every value collected above into a single baseline record. This baseline is referenced throughout the investigation and evidence sections of this report.

Table 4. Endpoint Baseline

Baseline Field	Actual Value
Hostname	DESKTOP-ELG9A9P
Username	ranbir (DESKTOP-ELG9A9P\ranbir)
Windows version	Windows 10 Pro, version 2009
OS architecture	64-bit
IPv4 address	192.168.0.183 (Ethernet0); gateway 192.168.0.1
CPU model / cores	13th Gen Intel Core i5-13420H; 2 cores / 2 logical processors
RAM (total)	Not captured in supplied evidence
Disk free space	C: 32,230,793,216 bytes free of 63,726,366,720 bytes
PowerShell version	5.1.19041.6456 (Desktop edition)
Windows Defender status	Windows Security screenshot shows no current threats; real-time protection setting is not shown
Administrator privilege confirmed	True (shown in supplied PowerShell output)
Baseline capture date/time	29 September 2026; screenshot shows 11:49:07 for the date command. Individual capture times vary or are not shown.

5. Troubleshooting

Table 5. Endpoint Setup Troubleshooting

Symptom	Diagnostic Steps
Command returns an error or nothing	Confirm PowerShell (not cmd.exe) is being used; re-run as Administrator; check for typos in cmdlet names.
No internet connectivity	Check VM network adapter mode (NAT/Bridged); verify host machine internet access; check hypervisor virtual network settings.
Event Viewer logs appear empty	Confirm the correct log (Application/Security/System) is selected; check that Windows Event Log service is running (`Get-Service EventLog`).

Symptom	Diagnostic Steps
Administrator check returns False	Re-launch PowerShell with “Run as Administrator”; confirm the logged-in account is a member of the local Administrators group.

6. Evidence to Record

- All values in the baseline table above (Section 1.13).
- Figures 1.1 through 1.12.

7. Security Relevance

A documented baseline lets an analyst distinguish expected, known-good endpoint state from anomalous state observed later in telemetry — the same principle used in real SOC environments when comparing an alert against a host's known configuration and normal behavior.

Module 2 — LimaCharlie EDR Deployment

2.1 LimaCharlie Introduction

LimaCharlie is a cloud-delivered EDR/SecOps platform: a lightweight sensor runs on the endpoint and streams telemetry to a cloud-hosted organization, where detection rules, investigation tools, and response actions are available through a web console and API.

What EDR means:

Endpoint Detection and Response — continuous endpoint telemetry collection combined with detection logic and response capability, as distinct from point-in-time antivirus scanning.

Why EDR is useful in SOC operations:

- Provides visibility into living-off-the-land techniques (e.g., PowerShell, cmd.exe) that traditional signature-based AV may not flag.
- Records rich context (parent/child process relationships, command lines, network destinations) needed for investigation.
- Supports custom, tunable detection logic rather than only vendor-supplied signatures.

Endpoint telemetry vs. traditional antivirus:

Traditional AV primarily evaluates files against known-bad signatures/heuristics at write or execution time. EDR telemetry continuously records behavior — what ran, what it was run by, what it connected to — enabling detections based on behavior patterns rather than only file identity.

How endpoint telemetry supports investigation:

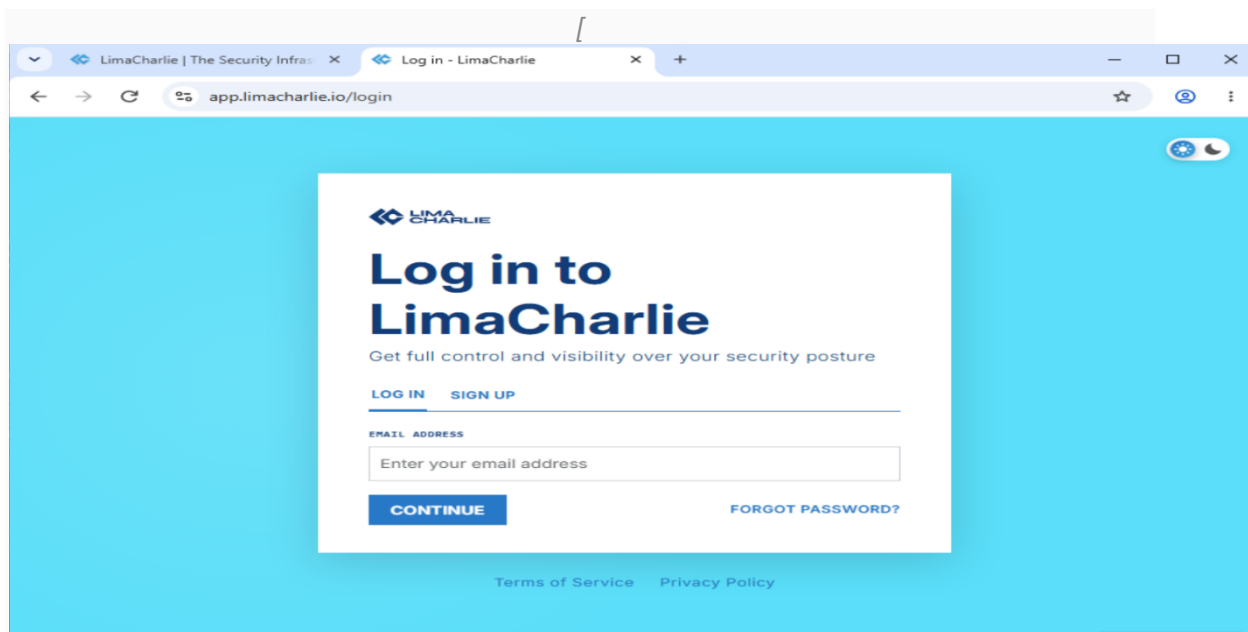
When an alert fires, the underlying telemetry (process tree, command line, user, network activity, timestamps) gives the analyst the context needed to determine whether the activity is malicious, benign, or requires further investigation.

2.2 Account and Organization Setup

Create or sign in to a LimaCharlie account at the LimaCharlie web console. Note: exact sign-up steps, field names, and console layout should be verified against the live LimaCharlie site at the time of the lab, since SaaS interfaces change over time.

Figure 2.1 — LimaCharlie Account / Login Screen

Purpose: Confirm access to the LimaCharlie web console before sensor deployment.



Evidence to capture:

- LimaCharlie login/dashboard URL bar
- Account or organization name (redact if sensitive)

Do NOT expose in this screenshot:

- Do not expose account credentials, API keys, or billing information.

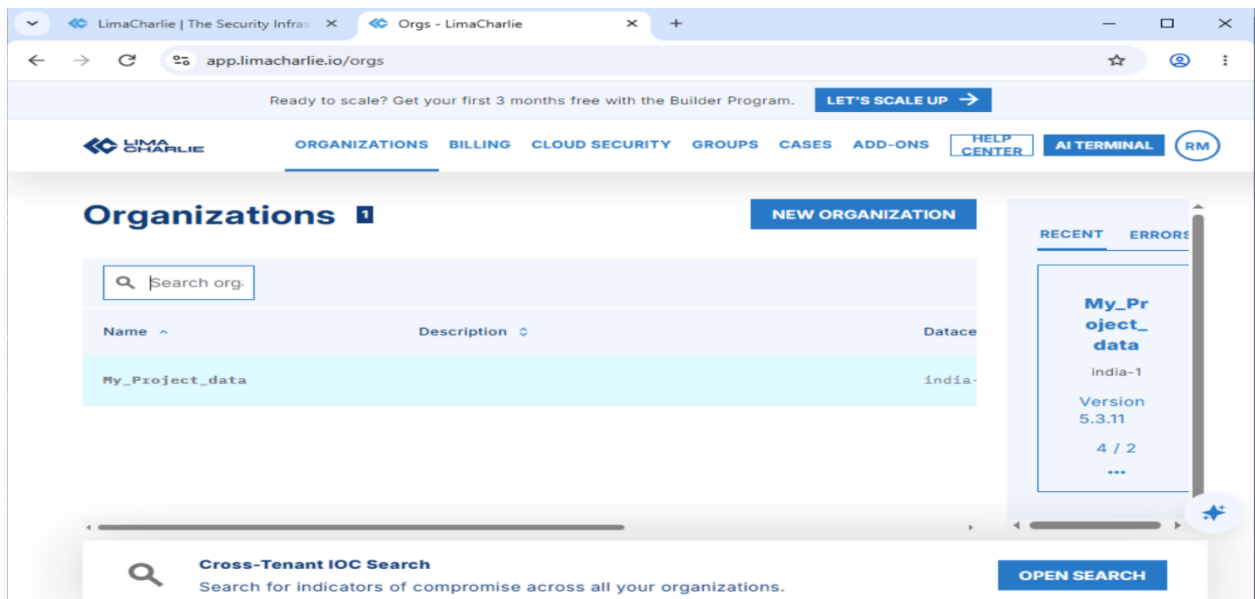
Observation: Not captured in supplied evidence for this figure.

2.3 Create / Select an Organization

In LimaCharlie, an “organization” (sometimes abbreviated “OID”) is the isolated tenant that holds your sensors, telemetry, and detection rules. Create a new organization dedicated to this lab (e.g., named after the project) or select an existing one reserved for testing. Keeping the lab in its own organization avoids mixing lab telemetry with any other environment.

Figure 2.2 — LimaCharlie Organization Created/Selected

Purpose: Confirm the lab is isolated inside its own LimaCharlie organization.



Evidence to capture:

- Organization name
- Organization ID (redact if this report will be shared publicly)

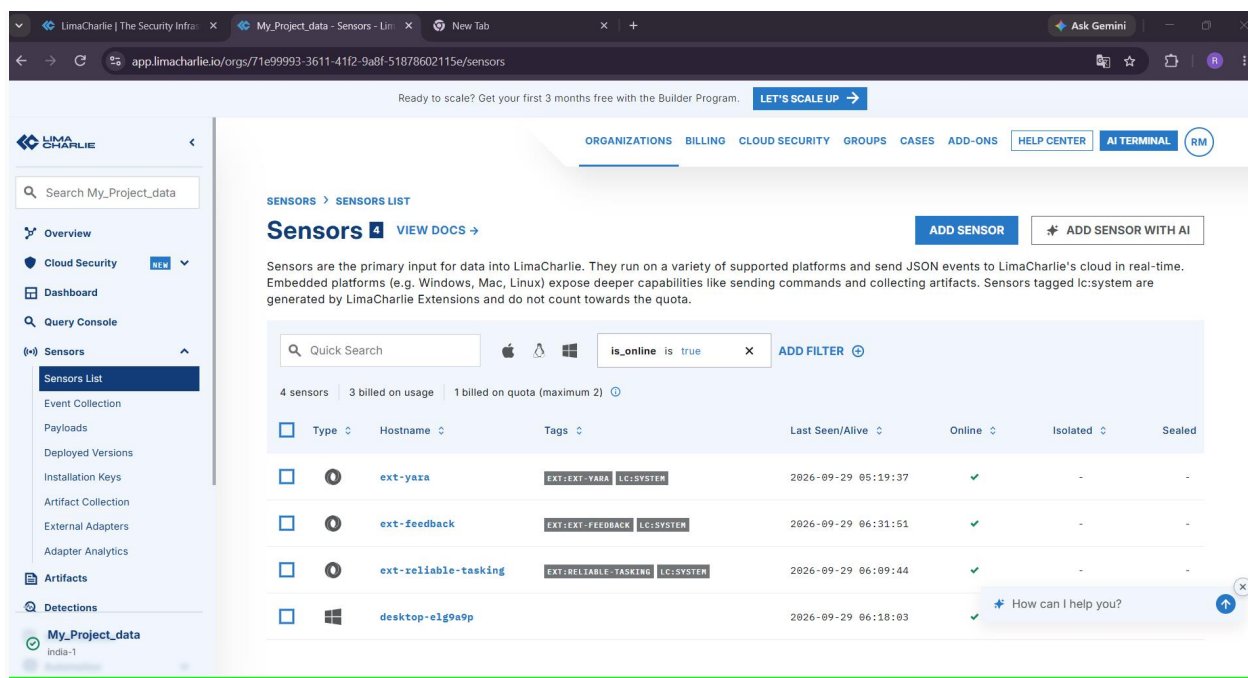
Observation: Not captured in supplied evidence for this figure.

2.4 Windows Sensor Deployment

From the LimaCharlie console, navigate to the sensor deployment / installation-keys section for your organization and generate a Windows installer/installation key. Exact menu wording should be verified in the live console.

Figure 2.3 — Sensor Installer / Installation Key Generated

Purpose: Confirm an installer or installation key has been generated for the Windows sensor.



Evidence to capture:

- Platform selected (Windows)
- Installation key identifier (redact the raw key value)

Do NOT expose in this screenshot:

- Never expose the raw installation key/secret in a screenshot that will be shared.

Observation: Not captured in supplied evidence for this figure.

2.5 Sensor Installation

Download

Download the Windows sensor installer/binary from the LimaCharlie console onto the monitored VM.

Installation

Run the installer with Administrator privileges. Follow the on-screen prompts, supplying the installation key generated in Section 2.4 when requested.

Permissions

The sensor requires Administrator/SYSTEM-level privileges to install as a Windows service and to collect kernel-level telemetry (process, network events).

Service

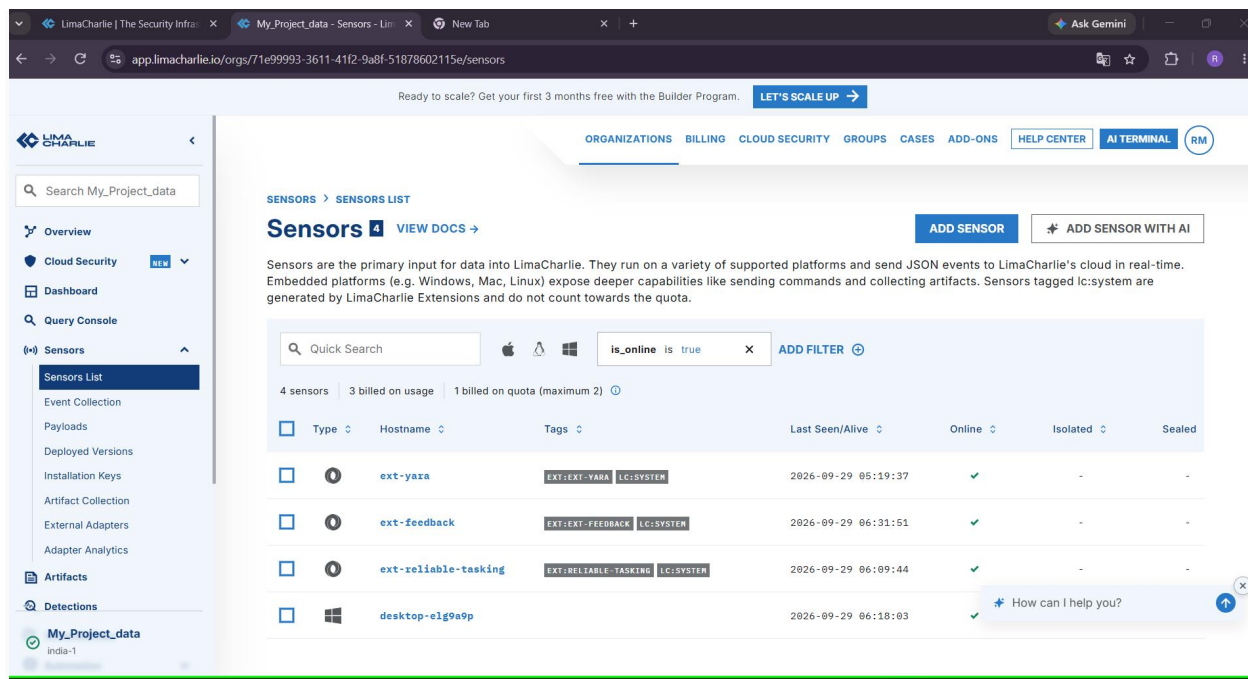
After installation, the sensor registers and runs as a Windows service, starting automatically on boot.

Connectivity

The sensor establishes an outbound connection to the LimaCharlie cloud platform over HTTPS; no inbound firewall rule is required.

Figure 2.4 — Sensor Installation in Progress / Complete

Purpose: Document the installation process for the Windows sensor.



Evidence to capture:

- Installer window
- Installation completion message

Observation: Not captured in supplied evidence for this figure.

2.6 Verify Sensor Service

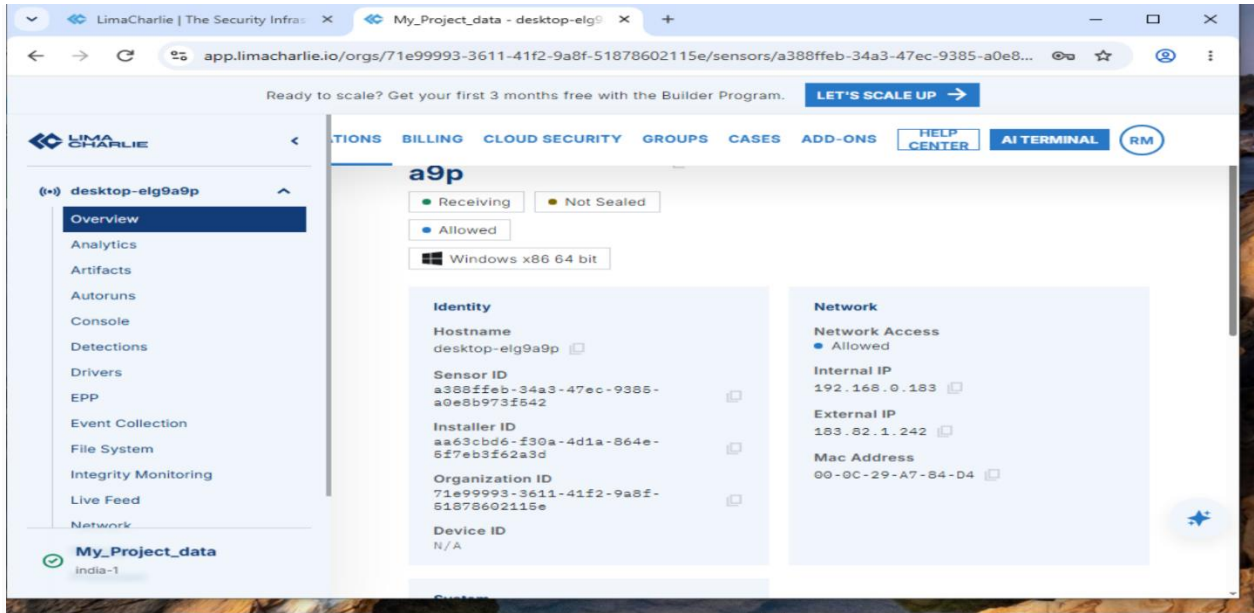
```
Get-Service | Where-Object { $_.DisplayName -like "*LimaCharlie*" }
```

Explanation: Confirms the LimaCharlie sensor is installed as a Windows service and reports its current running status. Exact service display name should be confirmed against the actual installation, as it may vary by sensor version.

Expected result: A service entry with Status = Running.

Figure 2.5 — LimaCharlie Sensor Service Status

Purpose: Confirm the sensor service is installed and running on the endpoint.



Evidence to capture:

- Service name
- Status column (Running)

Observation: Not captured in supplied evidence for this figure.

2.7 Verify LimaCharlie Console

In the LimaCharlie web console, navigate to the Sensors list for your organization and confirm:

- The endpoint appears in the sensor list.
- The sensor shows an Online/connected status.
- The reported hostname matches the baseline hostname recorded in Module 1.
- Telemetry (e.g., recent events) is arriving for the sensor.

Figure 2.6 — Endpoint Visible and Online in LimaCharlie Console

Purpose: Confirm the sensor is connected and telemetry is flowing from the monitored endpoint.

```
Administrator: Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> Get-Service | Where-Object { $_.DisplayName -like "*LimaCharlie*" }

Status   Name          DisplayName
-----
Running  rphcpsvc      LimaCharlie

PS C:\Windows\system32>
```

Evidence to capture:

- Sensor list entry
- Hostname
- Online/connected status
- Last-seen timestamp

Observation: Not captured in supplied evidence for this figure.

2.8 Sensor Troubleshooting

Table 6. Sensor Deployment Troubleshooting

Problem	Diagnostic Steps
Sensor offline	Check the Windows service status (Section 2.6); verify outbound HTTPS connectivity (Module 1.8); check installation logs.
No endpoint visible in console	Confirm the correct organization is selected in the console; re-verify the installation key used matched this organization.
Internet problem	Re-run the connectivity test in Module 1.8; check VM network adapter settings; check host-level firewall/proxy.
Permission problem	Re-install the sensor from an elevated (Administrator) prompt; confirm the installing account has local Administrator rights.
Service not running	Attempt to start the service manually (`Start-Service`); check Windows Event Viewer Application log for service start errors.
Incorrect organization	Uninstall and reinstall the sensor using an installation key generated from the intended organization.
Firewall / network restriction	Confirm outbound HTTPS (443) is not blocked by local firewall, VM host firewall, or network security group rules.

Module 3 — Endpoint Telemetry Verification

3.1 What Is Telemetry?

Telemetry is the raw stream of endpoint activity records the sensor reports: process creation and termination, command-line arguments, parent/child process relationships, user context, network connections, file activity, and timestamps. Detection rules are evaluated against this stream; investigation is performed by querying and reading this stream.

The categories below describe the telemetry this project is designed to collect. The supplied screenshots include selected process, network, and live-feed views; they do not provide a complete event record for each category.

- Processes — process name, PID, and start time.
- Parent/child relationships — which process launched which.
- Command lines — the full command-line string used to launch a process.
- Users — the account context a process ran under.
- Network connections — destination host/IP, port, and protocol associated with a process.
- Files — file-related events (created/modified) where relevant.
- Events and timestamps — the raw event stream with precise timing.
- Endpoint identity — hostname and sensor ID tying all telemetry back to this specific VM.

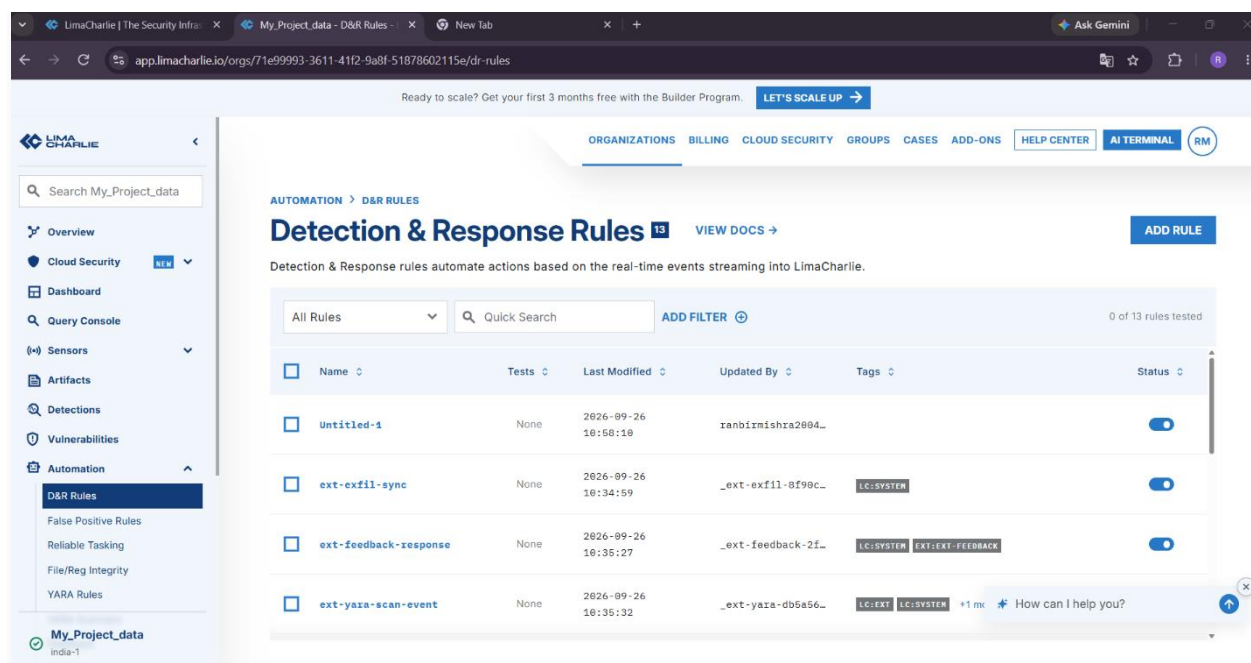
3.3 Telemetry Verification Checklist

In the LimaCharlie console, open the Timeline / Events view for the sensor and confirm each of the following is visible for a basic action (e.g., opening PowerShell):

- A process-creation event appears for the new process.
- The parent process is correctly identified.
- The full command line is visible and readable.
- The user context is correctly attributed.
- A timestamp is present and consistent with wall-clock time.

Figure 3.1 — LimaCharlie Timeline / Event Stream View

Purpose: Confirm raw telemetry (process, user, timestamp) is visible for the endpoint.



Evidence to capture:

- Timeline view
- At least one process-creation event with command line visible

Observation: Not captured in supplied evidence for this figure.

Figure 3.2 — Process Telemetry Detail

Purpose: Confirm process name, PID, and parent process are captured correctly.

```
PS C:\Windows\system32> powershell.exe -NoProfile -Command "Get-Process"
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
113	7	2684	7328	0.23	4688	0	AggregatorHost
349	21	7992	28316	0.22	9028	1	ApplicationFrameHost
172	10	6260	9788	2.05	9636	0	audiodg
1665	67	88836	171372	9.28	1056	1	chrome
596	29	95852	155536	4.25	1324	1	chrome
224	11	6936	9752	0.03	8232	1	chrome
473	39	21928	48348	3.34	8444	1	chrome
370	35	20588	55432	12.22	8452	1	chrome
186	14	8300	18008	0.09	8484	1	chrome
275	19	37520	72932	0.97	8496	1	chrome
266	21	43308	83648	0.64	8780	1	chrome
344	27	59044	98560	3.02	8884	1	chrome
449	46	159668	222780	15.77	9472	1	chrome
304	18	13344	24232	0.25	9584	1	chrome
208	17	20380	31580	0.14	10044	1	chrome
141	9	6560	12192	0.13	4580	0	conhost
271	13	4136	20460	0.34	11276	1	conhost
571	22	2224	5432	0.52	472	0	csrss
497	19	2148	5596	2.77	580	1	csrss
171	14	1688	5548	0.16	7844	2	csrss
402	16	3784	15856	0.22	7300	1	ctfmon
129	7	1220	5324	0.02	3332	0	dasHost
84	7	1084	5012	0.02	5672	1	DefenderSessionHelper
210	16	3204	10516	0.05	4768	0	dllhost
237	17	4388	12736	0.17	9184	1	dllhost
1026	47	38460	99628	7.36	1044	1	dwm
772	24	16492	40940	0.36	9340	2	dwm
137	9	2172	9680	0.05	11284	0	elevation_service
2209	85	221136	257104	11.95	6060	1	explorer
50	7	1548	3976	0.00	872	0	fontdrvhost
50	7	1768	4844	0.09	876	1	fontdrvhost
50	6	1412	4480	0.00	11704	2	fontdrvhost
0	0	60	8	0	0	0	Idle

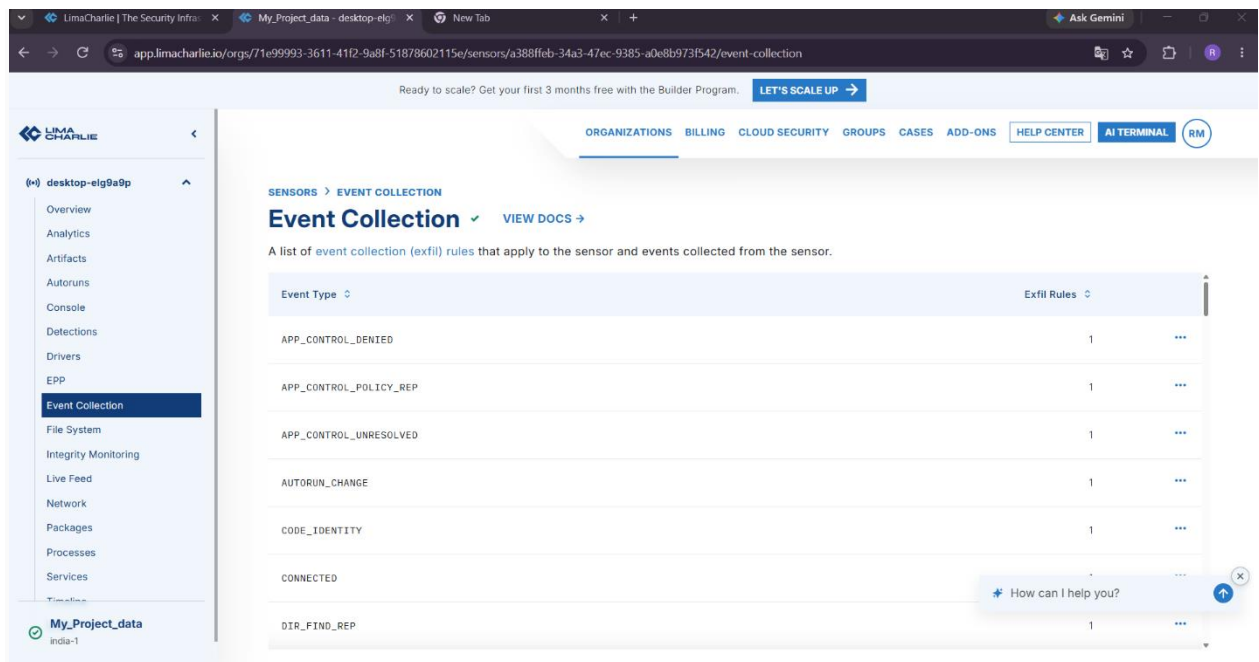
Evidence to capture:

- Process name
- PID
- Parent process name/PID

Observation: Not captured in supplied evidence for this figure.

Figure 3.3 — Network Telemetry Detail

Purpose: Confirm network-connection telemetry is captured and attributable to a process.



Evidence to capture:

- Source process
- Destination host/IP
- Destination port

Observation: Not captured in supplied evidence for this figure.

3.4 Telemetry Evidence Table

Table 7. Telemetry Evidence

Field	Actual Value
Hostname	Not captured in a correlated telemetry event
Username	Not captured in a correlated telemetry event
Process	Not captured in a correlated telemetry event
Parent Process	Not captured in a correlated telemetry event
PID	Not captured in a correlated telemetry event
Command Line	Not captured in a correlated telemetry event
Timestamp	Not captured in a correlated telemetry event
Network Destination	Not captured in a correlated telemetry event
Destination Port	Not captured in a correlated telemetry event

3.5 Troubleshooting: No Telemetry Visible

Table 8. Missing Telemetry Troubleshooting

Check	Action
Sensor online?	Re-verify sensor status in Module 2.7 before assuming a telemetry problem.
Correct sensor selected?	Confirm the console is viewing the sensor matching the baseline hostname, not a different/stale sensor entry.
Time range filter	Widen the console's time-range filter — telemetry may exist outside the default displayed window.
Activity actually generated?	Confirm the test action (e.g., opening PowerShell) was actually performed on the monitored VM, not another machine.

Module 4 — Detection Engineering

4.1 What Detection Engineering Means

Detection engineering is the discipline of translating a specific behavior of interest into a testable, tunable rule that a detection engine evaluates continuously against live telemetry. It differs from simply reading raw telemetry: a detection rule proactively flags matching activity as it occurs, generating an alert without requiring an analyst to manually search for it.

4.2 Telemetry vs. Detection

Telemetry (Module 3) is the raw recorded activity. Detection is the logic layer applied on top of that telemetry that decides which activity is noteworthy enough to surface as an alert. Without detection rules, an analyst would have to manually review the entire telemetry stream to find suspicious activity — impractical at scale.

4.3 Detection Logic, Alerts, and Severity

A detection rule expresses a condition (e.g., “process name equals powershell.exe”, or “parent process is cmd.exe AND child process is powershell.exe”) that, when matched against incoming telemetry, produces an alert. Each alert is assigned a severity (Section 4.4) reflecting how urgently it warrants analyst attention.

4.4 Severity Model

This project uses the following project-specific severity classification. These are the project's own classification criteria, not LimaCharlie's universal severity definitions, and are defined here so that every alert in this report is classified consistently.

Table 9. Severity Model

Severity	Meaning
LOW	Informational
MEDIUM	Suspicious
HIGH	Potential threat
CRITICAL	Immediate response required

Example escalation logic used in this project:

PowerShell execution	-> MEDIUM
Suspicious PowerShell + unusual network connection	-> HIGH
Confirmed malicious behavior in lab (never introduced)	-> CRITICAL

An alert may move from MEDIUM to HIGH when additional corroborating evidence is found during investigation — for example, PowerShell execution alone is only mildly suspicious, but PowerShell execution immediately followed by an unexpected outbound network connection from the same process is more suspicious and merits a higher severity.

4.5 False Positives and Tuning (Overview)

Any behavior-based rule risks flagging legitimate activity. Section 24 (False Positives) discusses this in depth; each detection below includes rule-specific false-positive notes and tuning recommendations.

4.6 Detection Rules

Note on rule syntax: LimaCharlie's Detection & Response (D&R) rule syntax should be verified directly against the current LimaCharlie documentation/console at the time this lab is performed, since exact field names and operators are platform-specific and may change between versions. The rule structures below describe the intended detection logic conceptually; the analyst must confirm and adapt the exact YAML/JSON syntax in the live console before saving a rule. The supplied rules page shows 0 of 13 rules tested; validate rule syntax and test results in the console before treating these detections as operational.

Detection 1 — PowerShell Execution

1. Detection objective:

Alert whenever powershell.exe is launched on the monitored endpoint.

2. Detection logic:

Trigger when a NEW_PROCESS event's file/process name equals powershell.exe (case-insensitive).

3. Required telemetry:

Process-creation (NEW_PROCESS) events including process name, PID, parent process, command line, and user.

4. Rule configuration:

```
# Conceptual D&R rule – verify exact LimaCharlie syntax before use
detect:
  event: NEW_PROCESS
  op: ends with
  path: event/FILE_PATH
  value: powershell.exe
respond:
  - action: report
    name: lab-powershell-execution
```

5. Test activity:

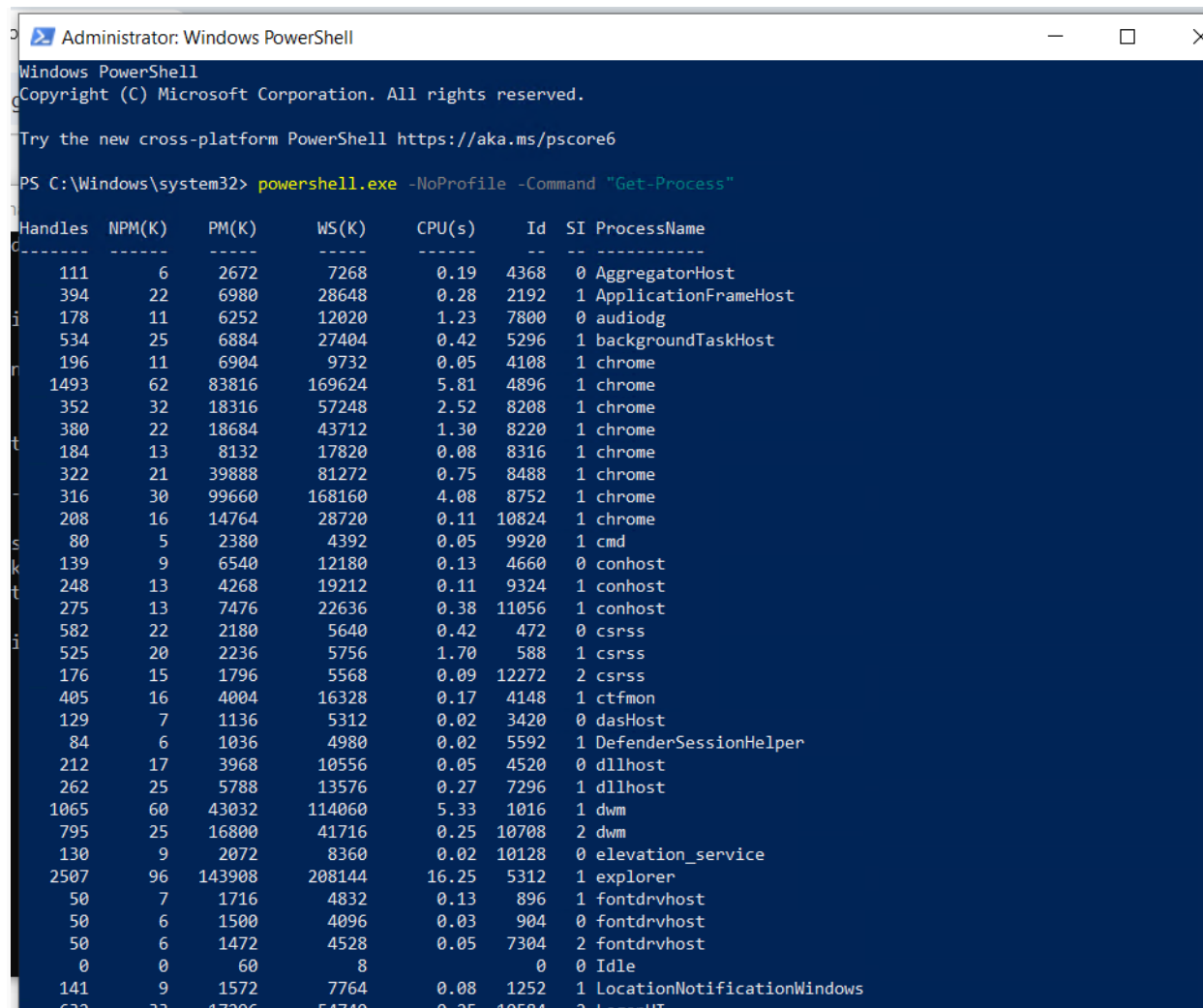
```
powershell.exe -NoProfile -Command "Get-Process"
```

6. Expected alert:

An alert named similarly to “lab-powershell-execution” should appear, referencing the monitored hostname and the powershell.exe process.

Figure DE.1 — Detection 1 Rule Configuration

Purpose: Document the saved PowerShell-execution detection rule.



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> powershell.exe -NoProfile -Command "Get-Process"

Handles  NPM(K)  PM(K)  WS(K)  CPU(s)  Id  SI ProcessName
-----  -
111      6      2672   7268   0.19    4368  0 AggregatorHost
394      22     6980   28648  0.28    2192  1 ApplicationFrameHost
178      11     6252   12020  1.23    7800  0 audiodg
534      25     6884   27404  0.42    5296  1 backgroundTaskHost
196      11     6904   9732   0.05    4108  1 chrome
1493     62    83816  169624  5.81    4896  1 chrome
352      32    18316  57248  2.52    8208  1 chrome
380      22    18684  43712  1.30    8220  1 chrome
184      13     8132   17820  0.08    8316  1 chrome
322      21    39888  81272  0.75    8488  1 chrome
316      30    99660  168160  4.08    8752  1 chrome
208      16    14764  28720  0.11   10824  1 chrome
80       5     2380   4392   0.05    9920  1 cmd
139      9     6540   12180  0.13    4660  0 conhost
248      13    4268   19212  0.11    9324  1 conhost
275      13    7476   22636  0.38   11056  1 conhost
582      22    2180   5640   0.42    472  0 csrss
525      20    2236   5756   1.70    588  1 csrss
176      15    1796   5568   0.09   12272  2 csrss
405      16    4004   16328  0.17    4148  1 ctfmon
129      7     1136   5312   0.02    3420  0 dasHost
84       6     1036   4980   0.02    5592  1 DefenderSessionHelper
212      17    3968   10556  0.05    4520  0 dllhost
262      25    5788   13576  0.27    7296  1 dllhost
1065     60    43032  114060  5.33    1016  1 dwm
795      25    16800  41716  0.25   10708  2 dwm
130      9     2072   8360   0.02   10128  0 elevation_service
2507     96   143908  208144  16.25   5312  1 explorer
50       7     1716   4832   0.13    896  1 fontdrvhost
50       6     1500   4096   0.03    904  0 fontdrvhost
50       6     1472   4528   0.05   7304  2 fontdrvhost
0        0        60      8        0  0 Idle
141      9     1572   7764   0.08   1252  1 LocationNotificationWindows
632     33    17296  54740  0.25   10584  2 LogonUI
```

Evidence to capture:

- Rule name
- Rule logic/condition

Observation: Not captured in supplied evidence for this figure.

7. Screenshot:

See Figure DE.1 above and Figure INC1 series in Incident 1.

8. Investigation:

Confirm parent process, command line, user, and timestamp for the triggering event (see Incident 1).

9. Severity:

MEDIUM by default per the severity model in Section 4.4.

10. MITRE ATT&CK mapping:

T1059.001 — Command and Scripting Interpreter: PowerShell.

11. False-positive considerations:

Administrators, software installers, and automation/scheduled scripts routinely invoke PowerShell. This detection alone should not be treated as conclusive evidence of compromise.

12. Tuning recommendations:

Consider excluding known-benign signed scripts/paths, or correlating with additional signals (unusual parent process, unusual command-line flags, unusual network activity) before escalating severity.

Detection 2 — cmd.exe Spawning PowerShell

1. Detection objective:

Alert when cmd.exe is the immediate parent process of powershell.exe — an ancestry pattern often associated with scripted or macro-triggered execution rather than direct interactive use.

2. Detection logic:

Trigger on NEW_PROCESS where process = powershell.exe AND parent process = cmd.exe.

3. Required telemetry:

Process-creation events with parent-process attribution.

4. Rule configuration:

```
# Conceptual D&R rule – verify exact LimaCharlie syntax before use
detect:
  event: NEW_PROCESS
  op: and
  rules:
    - op: ends with
      path: event/FILE_PATH
      value: powershell.exe
    - op: ends with
      path: event/PARENT/FILE_PATH
      value: cmd.exe
  respond:
    - action: report
      name: lab-cmd-spawns-powershell
```

5. Test activity:

From an open cmd.exe window, run:

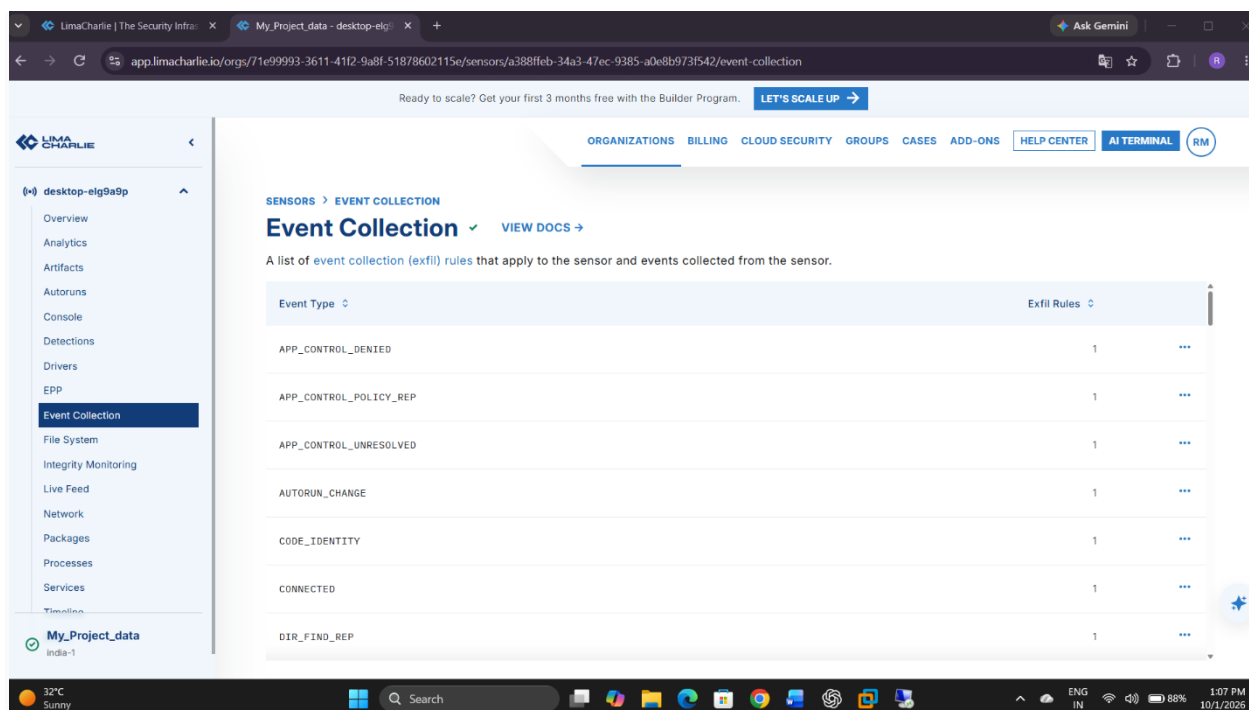
```
powershell.exe -NoProfile -Command "Get-Process"
```

6. Expected alert:

An alert reflecting the cmd.exe → powershell.exe ancestry.

Figure DE.2 — Detection 2 Rule Configuration

Purpose: Document the saved cmd-spawns-PowerShell detection rule.



Evidence to capture:

- Rule name
- Rule logic/condition

Observation: Not captured in supplied evidence for this figure.

7. Screenshot:

See Figure DE.2 above and Incident 2 figures.

8. Investigation:

Confirm the full process tree explorer.exe → cmd.exe → powershell.exe (Incident 2).

9. Severity:

MEDIUM, escalate to HIGH if combined with suspicious command-line content or network activity.

10. MITRE ATT&CK mapping:

T1059.001 (PowerShell) together with T1059.003 (Windows Command Shell).

11. False-positive considerations:

Many legitimate installers, batch scripts, and IT automation tools launch PowerShell from cmd.exe.

12. Tuning recommendations:

Baseline known automation tools that use this pattern in your environment and exclude their known, signed executables/paths.

Detection 3 — Scheduled Task Creation

1. Detection objective:

Detect creation of the lab's benign test scheduled task, representing the persistence technique category.

2. Detection logic:

Trigger on a process-creation event for schtasks.exe with a command line containing /create, or on a native Task Scheduler telemetry event if the sensor exposes one directly.

3. Required telemetry:

Process-creation events with full command-line capture for schtasks.exe.

4. Rule configuration:

```
# Conceptual D&R rule – verify exact LimaCharlie syntax before use
detect:
  event: NEW_PROCESS
  op: and
  rules:
    - op: ends with
      path: event/FILE_PATH
      value: schtasks.exe
    - op: contains
      path: event/COMMAND_LINE
      value: /create
  respond:
    - action: report
      name: lab-scheduled-task-creation
```

5. Test activity:

```
schtasks /create /tn "LimaCharlieLabTest" /tr "cmd.exe /c echo LabTest" /sc once /st 23:59
```

6. Expected alert:

An alert referencing schtasks.exe and the LimaCharlieLabTest task name.

Figure DE.3 — Detection 3 Rule Configuration

Purpose: Document the saved scheduled-task-creation detection rule.

```
PS C:\Users\ranbir> schtasks /create /tn "LimaCharlieLabTest" /tr "cmd.exe /c echo LabTest" /sc once /st 23:59
SUCCESS: The scheduled task "LimaCharlieLabTest" has successfully been created.
PS C:\Users\ranbir> █
```

Evidence to capture:

- Rule name
- Rule logic/condition

Observation: Not captured in supplied evidence for this figure.

7. Screenshot:

See Figure DE.3 above and Incident 3 figures.

8. Investigation:

Confirm the full schtasks.exe command line, the user context, and the resulting task (Incident 3).

9. Severity:

MEDIUM initially; scheduled-task creation is a common persistence technique so context (task name, target command, creating user) matters.

10. MITRE ATT&CK mapping:

T1053.005 — Scheduled Task/Job: Scheduled Task.

11. False-positive considerations:

Legitimate software installers and IT management tools frequently create scheduled tasks.

12. Tuning recommendations:

Exclude known-benign task names/publishers observed in your environment; flag tasks with unusual triggers (e.g., very short delay) or suspicious target commands for closer review.

Detection 4 — Suspicious Process-to-Network Activity

1. Detection objective:

Correlate a process (particularly an interpreter like powershell.exe) with an outbound network connection, to illustrate process-to-network correlation.

2. Detection logic:

Trigger on a NETWORK_CONNECTION event where the originating process is powershell.exe (or another interpreter of interest).

3. Required telemetry:

Network-connection events attributing destination host/IP and port to the originating process.

4. Rule configuration:

```
# Conceptual D&R rule – verify exact LimaCharlie syntax before use
detect:
  event: NETWORK_CONNECTIONS
  op: ends with
  path: event/PROCESS/FILE_PATH
  value: powershell.exe
respond:
  - action: report
    name: lab-powershell-network-activity
```

5. Test activity:

```
Test-NetConnection example.com -Port 443
nslookup example.com
```

6. Expected alert:

An alert reflecting a network connection attributed to powershell.exe.

Figure DE.4 — Detection 4 Rule Configuration

Purpose: Document the saved process-to-network correlation detection rule.

```
PS C:\Users\ranbir> Test-NetConnection example.com -Port 443
>> nslookup example.com
>>

ComputerName      : example.com
RemoteAddress     : 104.20.23.154
RemotePort        : 443
InterfaceAlias    : Ethernet0
SourceAddress     : 192.168.1.124
TcpTestSucceeded  : True

Server: dlinkrouter.local
Address: 192.168.1.1

Non-authoritative answer:
Name:   example.com
Addresses: 2606:4700:10::ac42:93f3
           2606:4700:10::6814:179a
           172.66.147.243
           104.20.23.154

PS C:\Users\ranbir> █
```

Evidence to capture:

- Rule name
- Rule logic/condition

Observation: Not captured in supplied evidence for this figure.

7. Screenshot:

See Figure DE.4 above and Incident 4 figures.

8. Investigation:

Confirm destination host/IP, port, protocol, and originating process (Incident 4).

9. Severity:

LOW/MEDIUM for a benign, known destination such as example.com; would only escalate with an unknown or reputationally suspicious destination.

10. MITRE ATT&CK mapping:

Mapped according to actual observed behavior — do not force a technique mapping onto benign connectivity-test traffic.

11. False-positive considerations:

PowerShell routinely makes legitimate network connections (e.g., module downloads, Windows Update related scripts, admin tooling).

12. Tuning recommendations:

Correlate destination reputation, connection volume/frequency, and beaconing patterns before treating this as more than informational.

Module 5 — Incident 1: Suspicious PowerShell

1. Objective

Generate, detect, investigate, map, and respond to a controlled PowerShell execution event.

2. Background

PowerShell is a common living-off-the-land execution vector; monitoring its use is a foundational EDR detection use case.

3. Prerequisites

- Modules 1–4 document endpoint setup, LimaCharlie deployment, and rule configuration; complete detection validation is not confirmed by the supplied evidence.
- Detection 1 (Section 4.6) enabled and saved in LimaCharlie.

4. Scenario

```
User
|
v
PowerShell
|
v
Suspicious/controlled command
|
v
LimaCharlie telemetry
|
v
Detection -> Alert -> Investigation -> MITRE ATT&CK -> Response
```

5. Step-by-Step Procedure

From the monitored VM, run the following safe, controlled command:

```
powershell.exe -NoProfile -Command "Get-Process"
```

This is intentionally benign — it only lists running processes — and is used purely to generate PowerShell-execution telemetry for the detection rule to evaluate. Do not substitute a destructive or credential-harvesting command.

6. Investigate

- Process name (powershell.exe)

- Parent process (e.g., explorer.exe or cmd.exe, depending on how it was launched)
- Command line (the full -NoProfile -Command "Get-Process" string)
- Username
- Timestamp
- Hostname
- PID
- Detection rule that fired
- Alert generated
- Related telemetry (subsequent events from the same PID, if any)

7. Figures

Figure INC1.1 — PowerShell Test Activity Executed

Purpose: Show the controlled PowerShell command being run on the endpoint.

```
PS C:\Users\ranbin> powershell.exe -NoProfile -Command "Get-Process"
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
112	7	3460	8120		4748	0	AggregatorHost
139	9	6532	12120		4652	0	conhost
273	13	4216	20196	0.72	5412	1	conhost
575	21	2180	5540		472	0	csrss
448	17	1832	5608		556	1	csrss
179	14	1776	5440		840	2	csrss
414	16	3820	15744	0.48	7076	1	ctfmon
129	7	1164	5372		3340	0	dasHost
84	6	1032	4960	0.02	5552	1	DefenderSessionHelper
228	17	3704	11744	0.16	3436	1	dllhost
217	17	3376	10940		4756	0	dllhost
962	44	32200	86900		1128	1	dwm
772	24	16768	41960		6656	2	dwm
2178	87	106468	179840	13.48	3144	1	explorer
50	6	1656	4540		844	1	fontdrvhost
50	6	1500	4048		852	0	fontdrvhost
50	6	1424	4304		4700	2	fontdrvhost
0	0	60	8		0	0	Idle
653	35	14504	56100		2736	2	LogonUI
1469	24	7784	19312		716	0	lsass
1165	57	37884	99612	1.66	10276	1	M365Copilot
0	0	388	50628		1680	0	Memory Compression
209	13	2632	1968		7352	0	MicrosoftEdgeUpdate
368	21	11456	30164		7568	0	MoUsoCoreWorker
491	17	9288	20648		3176	0	MpDefenderCoreService
318	17	10852	26284	0.16	1276	1	msedge
166	10	2768	10772	0.08	3500	1	msedge
202	13	7676	18936	0.05	9652	1	msedge
339	18	11912	38352	0.30	9824	1	msedge
1278	44	38984	123872	1.31	10272	1	msedge
376	22	14832	34684	0.67	588	1	msedgewebview2
205	12	8484	17136	0.16	4972	1	msedgewebview2
1169	47	37144	89756	2.08	6384	1	msedgewebview2
225	11	3136	13704	0.05	7944	1	msedgewebview2
367	20	12880	33452	0.84	8300	1	msedgewebview2
282	23	57316	58844	1.63	8980	1	msedgewebview2
1203	45	36056	114100	1.81	10788	1	msedgewebview2
175	11	2844	10708	0.03	10824	1	msedgewebview2
333	18	11920	40160	0.80	11060	1	msedgewebview2
357	22	17356	47104	1.22	11064	1	msedgewebview2

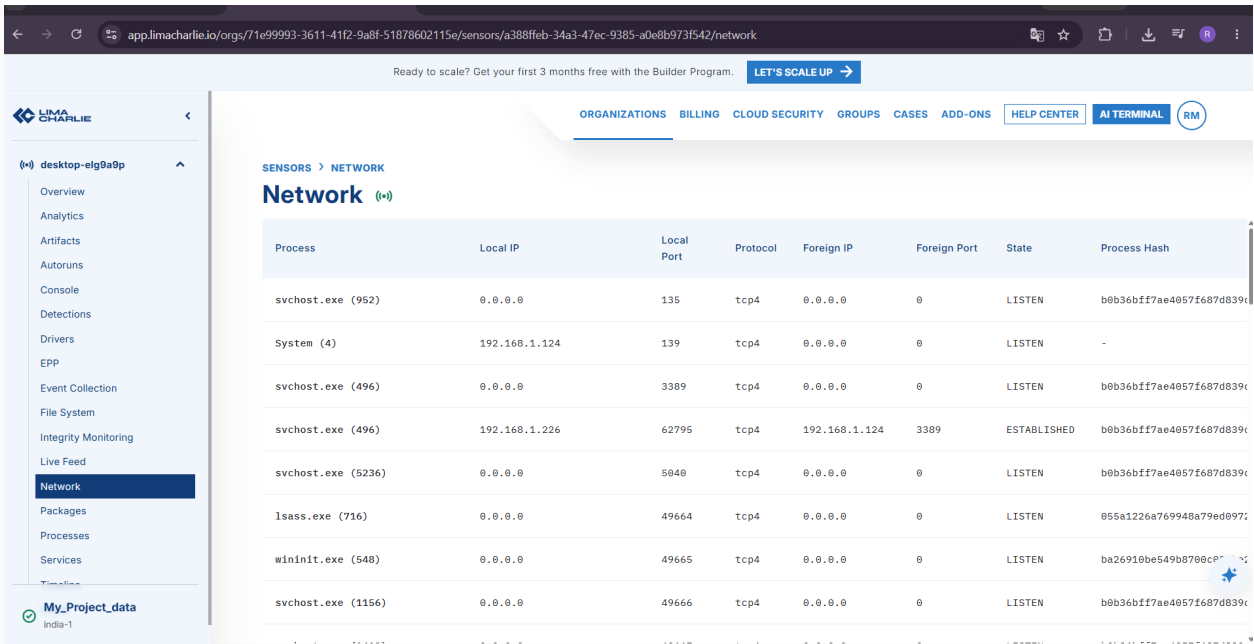
Evidence to capture:

- Terminal/PowerShell window
- Exact command executed
- Output of Get-Process

Observation: Not captured in supplied evidence for this figure.

Figure INC1.2 — Detection Rule Triggered

Purpose: Evidence that Detection 1 evaluated the telemetry and fired.



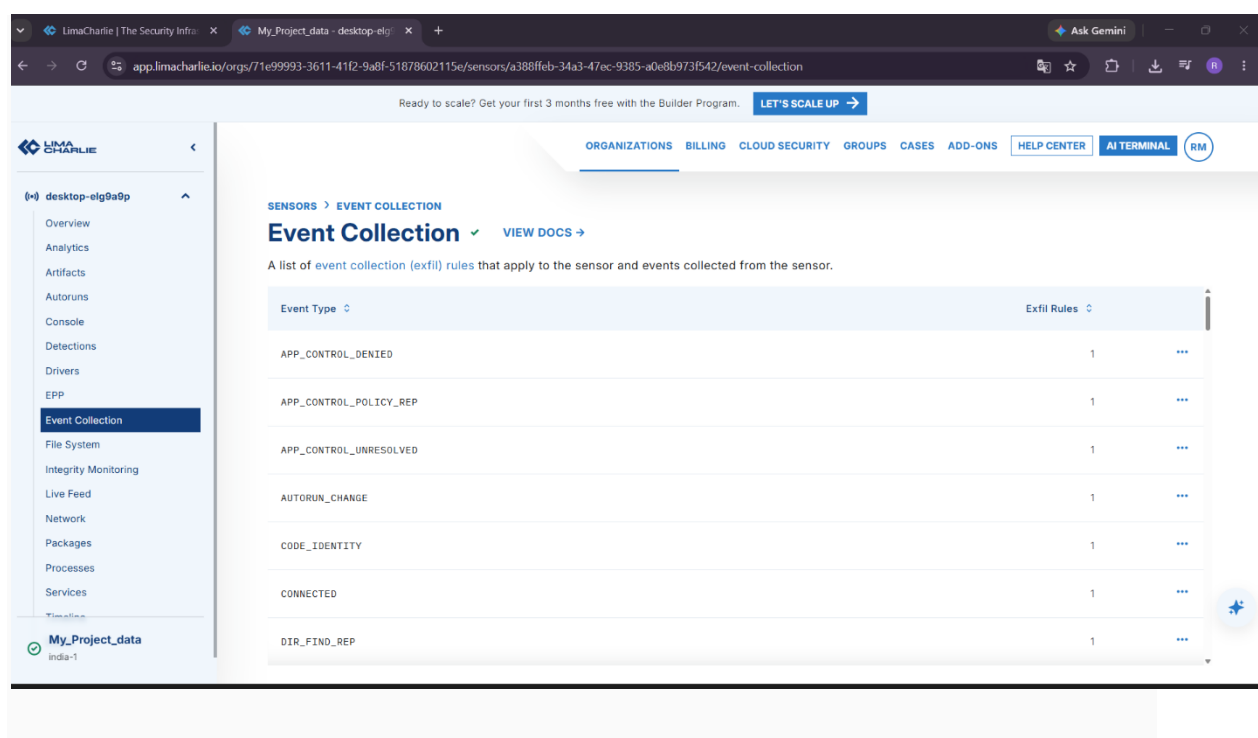
Evidence to capture:

- Rule name
- Triggering event reference

Observation: Not captured in supplied evidence for this figure.

Figure INC1.3 — Alert Generated

Purpose: Evidence of the resulting alert in the LimaCharlie console.



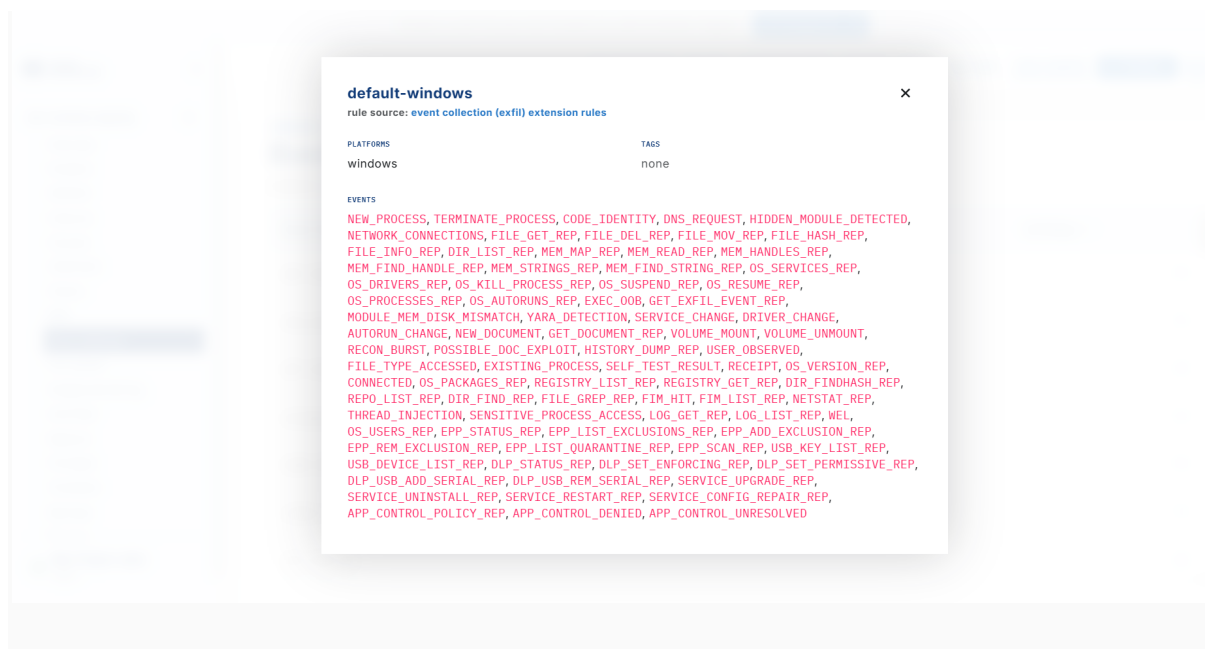
Evidence to capture:

- Alert name
- Endpoint/hostname
- Timestamp

Observation: Not captured in supplied evidence for this figure.

Figure INC1.4 — Process Detail

Purpose: Process name, PID, and execution context of the flagged PowerShell process.



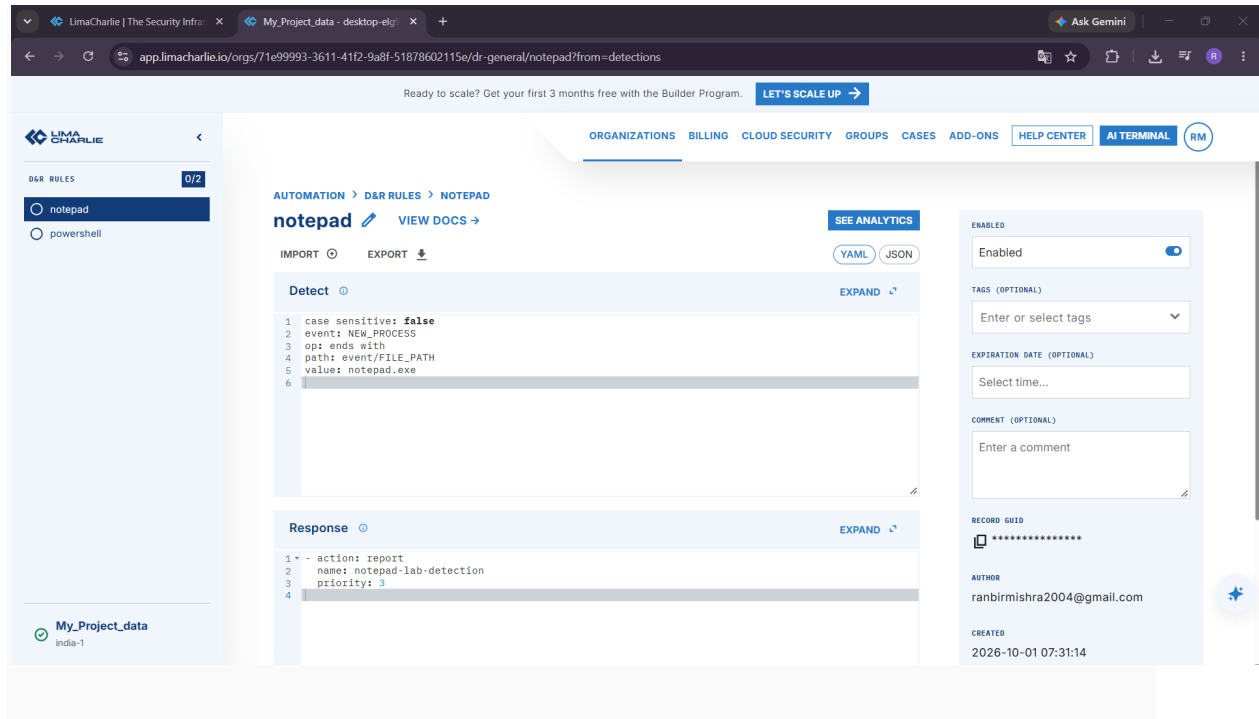
Evidence to capture:

- Process name
- PID
- Start time

Observation: Not captured in supplied evidence for this figure.

Figure INC1.5 — Command-Line Detail

Purpose: The full command line captured by telemetry for the flagged process.



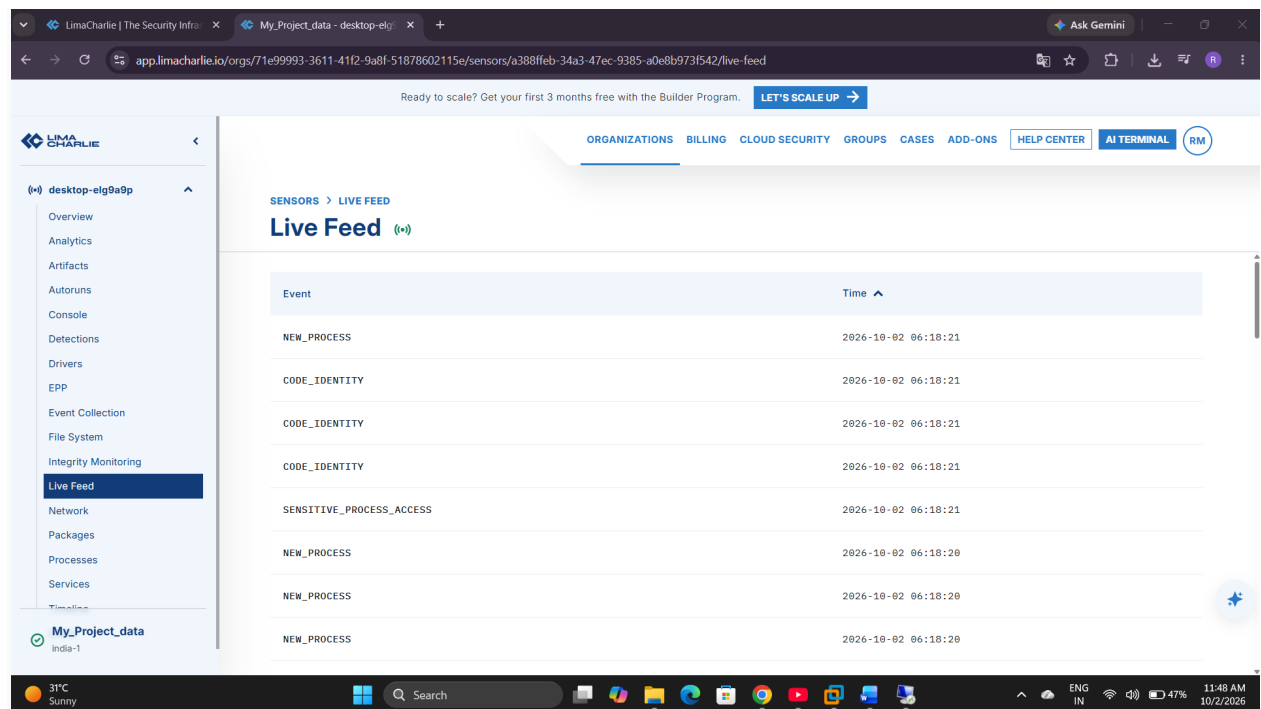
Evidence to capture:

- Complete command-line string

Observation: Not captured in supplied evidence for this figure.

Figure INC1.6 — Investigation View

Purpose: The analyst's investigation of parent process, user, and timeline context for this alert.



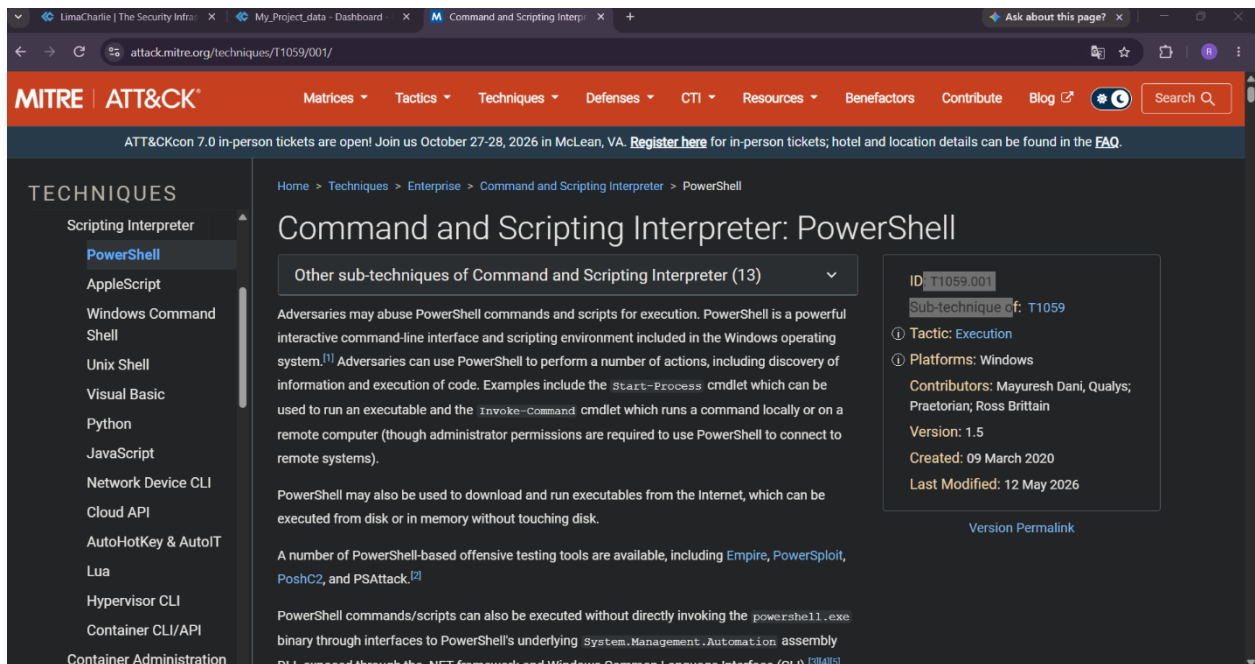
Evidence to capture:

- Parent process
- User
- Timeline entries around the event

Observation: Not captured in supplied evidence for this figure.

Figure INC1.7 — MITRE ATT&CK Mapping

Purpose: Mapping of this activity to T1059.001.



Evidence to capture:

- Technique ID and name
- Justification note

Observation: Not captured in supplied evidence for this figure.

Figure INC1.8 — Response Action

Purpose: The controlled response action performed (if any) for this incident.

No matching screenshot was supplied for this evidence item.

Evidence to capture:

- Response action taken
- Result/confirmation

Observation: Not captured in supplied evidence for this figure.

Figure INC1.9 — Verification

Purpose: Confirmation that the controlled test activity has concluded/stopped.

No matching screenshot was supplied for this evidence item.

Evidence to capture:

- Process no longer running
- No further related alerts

Observation: Not captured in supplied evidence for this figure.

8. MITRE ATT&CK Mapping

T1059.001 — Command and Scripting Interpreter: PowerShell.

9. Incident Evidence Table

Table 10. Incident 1 Evidence

Evidence	Value
Incident ID	Not captured in supplied evidence
Hostname	Not captured in supplied evidence
Username	Not captured in supplied evidence
Timestamp	Not captured in supplied evidence
Process	Not captured in supplied evidence
Parent Process	Not captured in supplied evidence
Child Process	Not captured in supplied evidence
PID	Not captured in supplied evidence
Command Line	Not captured in supplied evidence
Network Connection	Not captured in supplied evidence
Detection Rule	Not captured in supplied evidence
Alert	Not captured in supplied evidence
ATT&CK Technique	Not captured in supplied evidence
Severity	Not captured in supplied evidence
Response Action	Not captured in supplied evidence
Verification	Not captured in supplied evidence

10. Incident Timeline

Table 11. Incident 1 Timeline

Time	Event	Evidence	Analyst Observation
Not captured	Activity generated	Not captured	No incident-specific evidence captured.
Not captured	Telemetry received	Not captured	No correlated telemetry record supplied.
Not captured	Detection triggered	Not captured	No rule firing shown; the D&R page reports 0 of 13 rules tested.
Not captured	Alert investigated	Not captured	No alert investigation captured.
Not captured	ATT&CK mapped	Not captured	No incident-specific mapping evidence captured.
Not captured	Response performed	Not captured	No response action captured.
Not captured	Activity verified stopped	Not captured	No verification evidence captured.

11. Security Relevance

PowerShell execution monitoring is a baseline EDR capability because interpreter-based execution is widely used both legitimately and by adversaries; consistent visibility here is foundational to a broader detection strategy.

Module 6 — Incident 2: Suspicious Process Chain

1. Objective

Generate and investigate a controlled, unusual parent-child process chain.

2. Background

Process ancestry (which process launched which) is one of the most valuable pieces of EDR context: the same target process (powershell.exe) launched directly by a user versus launched by cmd.exe or another script host carries very different investigative weight.

3. Scenario

```
explorer.exe
|
+-- cmd.exe
    |
    +-- powershell.exe
```

4. Step-by-Step Procedure

- From the Windows desktop, open File Explorer/Start (explorer.exe context).
- Launch cmd.exe.
- From within cmd.exe, launch powershell.exe (e.g., simply typing `powershell` and pressing Enter, or re-running the Section 5 command from Incident 1).

5. Investigate

- Parent process (cmd.exe) and its own parent (explorer.exe)
- Child process (powershell.exe)
- Process ID for each process in the chain
- Command line for each process
- User context
- Timestamp for each process-creation event
- Full process tree as rendered by LimaCharlie
- Detection rule (Detection 2) that fired

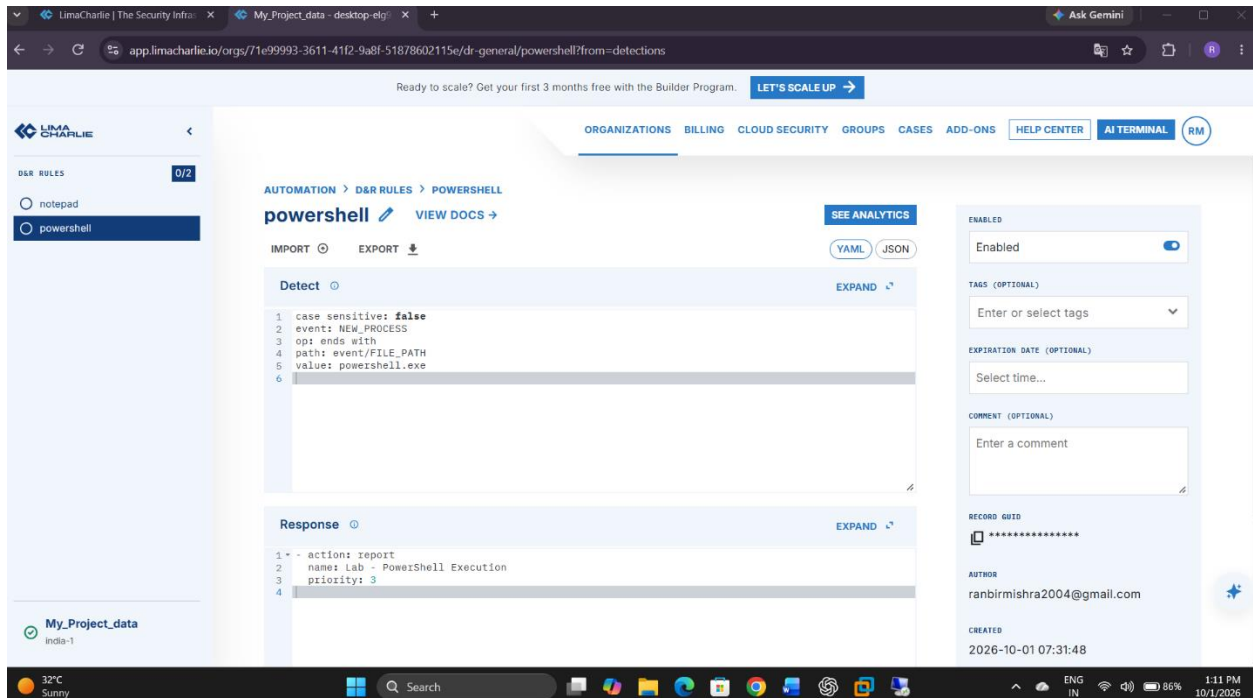
6. Why Process Ancestry Matters

An EDR investigation that only shows “powershell.exe ran” is far less useful than one that shows the full ancestry chain, because the same terminal process reached through different parents implies very different scenarios — a user manually opening a PowerShell window versus a script, macro, or scheduled job spawning it programmatically. Ancestry is often the single strongest signal for distinguishing benign interactive use from scripted or automated execution.

7. Figures

Figure INC2.1 — Process Chain Generated

Pur



pose: Show cmd.exe being launched and then powershell.exe launched from within it.

Evidence to capture:

- cmd.exe window
- powershell.exe launched from inside it

Observation: Not captured in supplied evidence for this figure.

Figure INC2.2 — Detection Rule Triggered (cmd->PowerShell)

Purpose: Evidence that Detection 2 evaluated and fired on this ancestry pattern.

The screenshot shows the LimaCharlie web interface with the 'Network' sensor selected. The table displays the following data:

Process	Local IP	Local Port	Protocol	Foreign IP	Foreign Port	State	Process Hash
svchost.exe (1000)	0.0.0.0	135	tcp4	0.0.0.0	0	LISTEN	b0b36bff7ae4057f687d839c
System (4)	192.168.1.124	139	tcp4	0.0.0.0	0	LISTEN	-
svchost.exe (464)	0.0.0.0	3389	tcp4	0.0.0.0	0	LISTEN	b0b36bff7ae4057f687d839c
svchost.exe (464)	192.168.1.226	10111	tcp4	192.168.1.124	3389	ESTABLISHED	b0b36bff7ae4057f687d839c
svchost.exe (6024)	0.0.0.0	5840	tcp4	0.0.0.0	0	LISTEN	b0b36bff7ae4057f687d839c
lsass.exe (740)	0.0.0.0	49664	tcp4	0.0.0.0	0	LISTEN	055a1226a7e9948a79ed0972
wininit.exe (580)	0.0.0.0	49665	tcp4	0.0.0.0	0	LISTEN	ba26910be549b870c0P...2
svchost.exe (1292)	0.0.0.0	49666	tcp4	0.0.0.0	0	LISTEN	b0b36bff7ae4057f687d839c

Evidence to capture:

- Rule name
- Triggering event reference

Observation: Not captured in supplied evidence for this figure.

Figure INC2.3 — Full Process Tree in LimaCharlie

Purpose: The rendered process tree showing explorer.exe -> cmd.exe -> powershell.exe.

No matching screenshot was supplied for this evidence item.

Evidence to capture:

- All three processes in the tree
- PIDs for each

Observation: Not captured in supplied evidence for this figure.

Figure INC2.4 — Command Lines for Each Process in the Chain

Purpose: Command-line evidence for cmd.exe and powershell.exe.



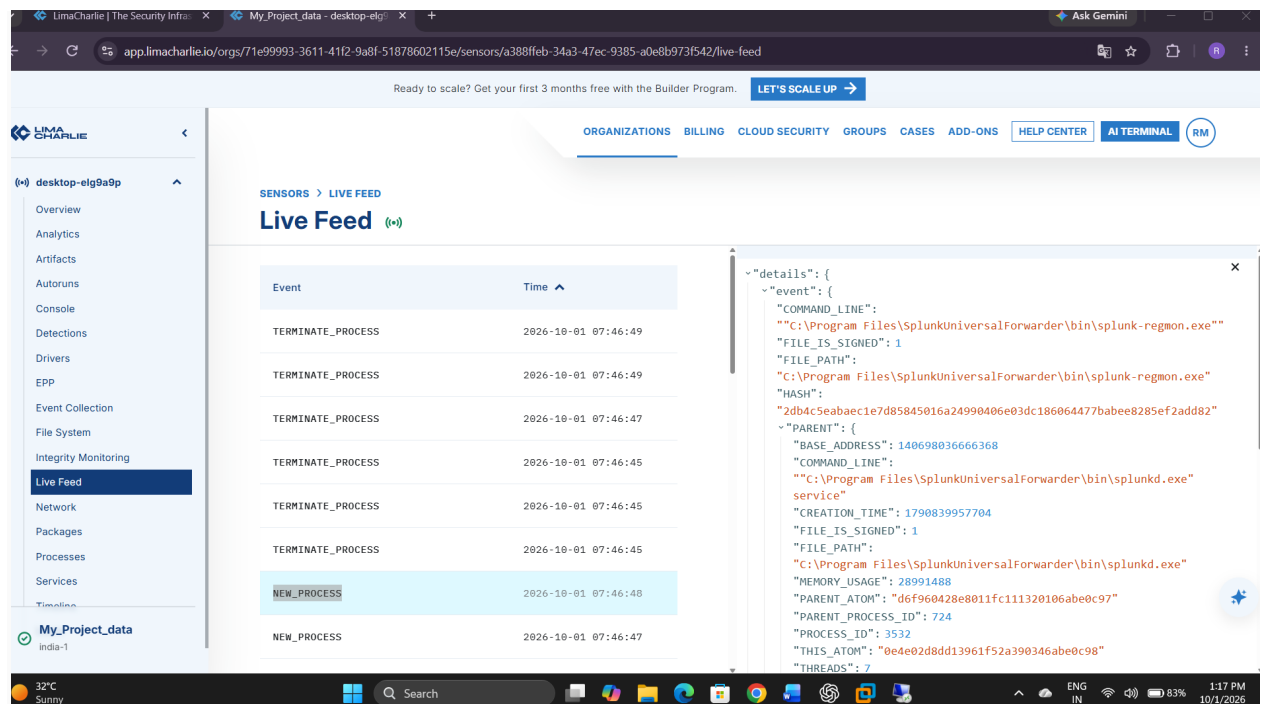
Evidence to capture:

- cmd.exe command line
- powershell.exe command line

Observation: Not captured in supplied evidence for this figure.

Figure INC2.5 — Investigation Summary

Purpose: Analyst notes correlating user, timestamps, and ancestry for this alert.



Evidence to capture:

- User

- Timestamps
- Ancestry summary

Observation: Not captured in supplied evidence for this figure.

8. MITRE ATT&CK Mapping

Map only what is supported by the observed activity: T1059.001 (PowerShell) and T1059.003 (Windows Command Shell) for the two interpreters involved. Do not add a persistence, defense-evasion, or lateral-movement technique unless the actual observed activity supports it.

9. Incident Evidence Table

Table 12. Incident 2 Evidence

Evidence	Value
Incident ID	Not captured in supplied evidence
Hostname	Not captured in supplied evidence
Username	Not captured in supplied evidence
Timestamp	Not captured in supplied evidence
Process	Not captured in supplied evidence
Parent Process	Not captured in supplied evidence
Child Process	Not captured in supplied evidence
PID	Not captured in supplied evidence
Command Line	Not captured in supplied evidence
Network Connection	Not captured in supplied evidence
Detection Rule	Not captured in supplied evidence
Alert	Not captured in supplied evidence
ATT&CK Technique	Not captured in supplied evidence
Severity	Not captured in supplied evidence
Response Action	Not captured in supplied evidence
Verification	Not captured in supplied evidence

10. Incident Timeline

Table 13. Incident 2 Timeline

Time	Event	Evidence	Analyst Observation
Not captured	Activity generated	Not captured	No incident-specific evidence captured.
Not captured	Telemetry received	Not captured	No correlated telemetry record supplied.
Not captured	Detection triggered	Not captured	No rule firing shown; the D&R page reports 0 of 13 rules tested.

Time	Event	Evidence	Analyst Observation
Not captured	Alert investigated	Not captured	No alert investigation captured.
Not captured	ATT&CK mapped	Not captured	No incident-specific mapping evidence captured.
Not captured	Response performed	Not captured	No response action captured.
Not captured	Activity verified stopped	Not captured	No verification evidence captured.

11. Security Relevance

Detecting abnormal process ancestry is a core EDR use case for surfacing scripted, automated, or macro-triggered execution that would otherwise look identical to normal PowerShell use at the single-process level.

Module 7 — Incident 3: Persistence Detection

1. Objective

Create, detect, investigate, map, and safely remove a benign scheduled-task persistence artifact.

2. Background

Scheduled tasks are a common, low-friction persistence mechanism. This module uses a harmless scheduled task — one that only echoes a string — to demonstrate the full detect/investigate/respond/cleanup cycle without introducing any actual persistence risk.

3. Workflow

```
Scheduled Task Created
|
v
LimaCharlie Event
|
v
Detection
|
v
Investigation
|
v
ATT&CK Mapping
|
v
Remove Lab Task
```

4. Step-by-Step Procedure

Create the benign test task:

```
schtasks /create /tn "LimaCharlieLabTest" /tr "cmd.exe /c echo LabTest" /sc once /st 23:59
```

Explanation: Creates a one-time scheduled task named “LimaCharlieLabTest” that, if it ever ran, would only print the text “LabTest” and exit — no persistence risk, no payload. Adjust the /st (start time) value as needed for your actual system clock; the goal is simply to generate a scheduled-task-creation event, not to let the task execute.

Confirm creation:

```
schtasks /query /tn "LimaCharlieLabTest"
```

5. Investigate

- The schtasks.exe process-creation event and its full command line
- The user context that created the task
- The resulting Task Scheduler entry (name, trigger, target command)

- The corresponding LimaCharlie detection/alert (Detection 3)
- Timestamp of creation

6. Figures

Figure INC3.1 — Scheduled Task Creation Command

Purpose: Show the schtasks /create command being run.

```
C:\Windows\system32>schtasks /create /tn "LimaCharlieLabTest" /tr "cmd.exe /c echo LabTest" /sc once /st 23:59
SUCCESS: The scheduled task "LimaCharlieLabTest" has successfully been created.

C:\Windows\system32>
```

Evidence to capture:

- Terminal window
- Full command executed
- Success confirmation message

Observation: Not captured in supplied evidence for this figure.

Figure INC3.2 — Detection Rule Triggered (Scheduled Task)

Purpose: Evidence that Detection 3 fired on the schtasks.exe activity.

The screenshot shows the LimaCharlie web interface. On the left is a sidebar with navigation options like Overview, Analytics, Artifacts, etc. The main area is titled 'Live Feed' and contains a table of events. The table has columns for 'Event' and 'Time'. The events listed are 'TERMINATE_PROCESS' and 'NEW_PROCESS'. The 'NEW_PROCESS' event is highlighted in blue. To the right of the table, a 'details' panel is open, showing a JSON object with event details. The details include 'COMMAND_LINE', 'FILE_PATH', 'HASH', 'PARENT' information, and 'PROCESS_ID'.

Event	Time
TERMINATE_PROCESS	2026-10-01 07:46:49
TERMINATE_PROCESS	2026-10-01 07:46:49
TERMINATE_PROCESS	2026-10-01 07:46:47
TERMINATE_PROCESS	2026-10-01 07:46:45
TERMINATE_PROCESS	2026-10-01 07:46:45
TERMINATE_PROCESS	2026-10-01 07:46:45
NEW_PROCESS	2026-10-01 07:46:48
NEW_PROCESS	2026-10-01 07:46:47

```
{
  "details": {
    "event": {
      "COMMAND_LINE": "C:\\Program Files\\SplunkUniversalForwarder\\bin\\splunk-regmon.exe",
      "FILE_IS_SIGNED": 1,
      "FILE_PATH": "C:\\Program Files\\SplunkUniversalForwarder\\bin\\splunk-regmon.exe",
      "HASH": "2db4c5eabae1e7d85845016a24990406e03dc186064477babe8285ef2add82",
      "PARENT": {
        "BASE_ADDRESS": 14069803666368,
        "COMMAND_LINE": "C:\\Program Files\\SplunkUniversalForwarder\\bin\\splunkd.exe",
        "CREATION_TIME": 1790839957704,
        "FILE_IS_SIGNED": 1,
        "FILE_PATH": "C:\\Program Files\\SplunkUniversalForwarder\\bin\\splunkd.exe",
        "MEMORY_USAGE": 28991488,
        "PARENT_ATOM": "d6f960428e8011fc111320106abe0c97",
        "PARENT_PROCESS_ID": 724,
        "PROCESS_ID": 3532,
        "THIS_ATOM": "0e4e02d8dd13961f52a390346abe0c98",
        "THREADS": 7
      }
    }
  }
}
```

Evidence to capture:

- Rule name
- Triggering event reference

Observation: Not captured in supplied evidence for this figure.

Figure INC3.3 — Alert and Task Detail in LimaCharlie

Purpose: The alert showing the created task name and target command.

```
C:\Windows\system32>schtasks /query /tn "LimaCharlieLabTest"

Folder: \
TaskName                               Next Run Time           Status
-----
LimaCharlieLabTest                     01-10-2026 23:59:00     Ready
```

Evidence to capture:

- Task name (LimaCharlieLabTest)
- Target command
- Creating user

Observation: Not captured in supplied evidence for this figure.

Figure INC3.4 — Task Scheduler Confirmation

Purpose: Native Windows confirmation that the task exists.

```
C:\Windows\system32>schtasks /delete /tn "LimaCharlieLabTest" /f
SUCCESS: The scheduled task "LimaCharlieLabTest" was successfully deleted.
```

Evidence to capture:

- Task Scheduler or schtasks /query output
- Task name and trigger

Observation: Not captured in supplied evidence for this figure.

Figure INC3.5 — Task Removal (Cleanup)

Purpose: Evidence the lab artifact was removed after the demonstration.

```
C:\Windows\system32>schtasks /query /tn "LimaCharlieLabTest"
ERROR: The system cannot find the file specified.
```

Evidence to capture:

- schtasks /delete command
- Success confirmation

Observation: Not captured in supplied evidence for this figure.

Figure INC3.6 — Post-Cleanup Verification

Purpose: Confirmation the task no longer exists after cleanup.

```
PS C:\Users\ranbir> schtasks /delete /tn "LimaCharlieLabTest" /f
SUCCESS: The scheduled task "LimaCharlieLabTest" was successfully deleted.
PS C:\Users\ranbir> █
```

Evidence to capture:

- schtasks /query showing task not found, or equivalent

Observation: Not captured in supplied evidence for this figure.

7. Cleanup

After the demonstration is fully documented, remove the lab task:

```
schtasks /delete /tn "LimaCharlieLabTest" /f
```

Verify removal by re-running the /query command from Section 4 and confirming the task is no longer found.

8. MITRE ATT&CK Mapping

T1053.005 — Scheduled Task/Job: Scheduled Task.

9. Incident Evidence Table

Table 14. Incident 3 Evidence

Evidence	Value
Incident ID	Not assigned in supplied evidence
Hostname	Not captured in the task screenshots
Username	Not captured in the task screenshots
Timestamp	Creation time not captured; query showed next run at 1 October 2026, 23:59
Process	schtasks.exe (creation and deletion commands shown)
Parent Process	Not captured in supplied evidence
Child Process	Not observed running; task action was configured to invoke cmd.exe /c echo LabTest
PID	Not captured in supplied evidence
Command Line	schtasks /create /tn "LimaCharlieLabTest" /tr "cmd.exe /c echo LabTest" /sc once /st 23:59
Network Connection	Not applicable to this task test
Detection Rule	No matching rule test or firing is shown
Alert	Not captured in supplied evidence
ATT&CK Technique	T1053.005 — Scheduled Task/Job: Scheduled Task (behavior mapping; benign lab activity)
Severity	No severity assigned; benign controlled test
Response Action	Task removed with schtasks /delete /tn "LimaCharlieLabTest" /f
Verification	Subsequent schtasks /query reported that the task could not be found

10. Incident Timeline

Table 15. Incident 3 Timeline

Time	Event	Evidence	Analyst Observation
Not captured	Activity generated	Successful schtasks /create command	The task was created successfully; query showed status Ready.

Time	Event	Evidence	Analyst Observation
Not captured	Telemetry received	Not captured	No LimaCharlie event correlated to task creation was supplied.
Not captured	Detection triggered	Not captured	No task detection or alert is shown.
Not captured	Alert investigated	Not captured	No alert investigation captured.
Not captured	ATT&CK mapped	T1053.005	Scheduled-task creation behavior; benign test, not evidence of adversary activity.
Not captured	Response performed	Successful schtasks /delete command	The named test task was deleted.
Not captured	Activity verified stopped	Subsequent query: task not found	Cleanup is confirmed; exact action times are not captured.

11. Security Relevance

Scheduled-task creation is one of the most common real-world persistence techniques; detecting and investigating it — including safely removing the artifact afterward — mirrors a genuine eradication step in the incident-response lifecycle.

Module 8 — Incident 4: Suspicious Network Activity

1. Objective

Generate controlled outbound network activity and investigate the resulting process-to-network telemetry.

2. Background

Correlating a process with the network destinations it contacts is essential for identifying command-and-control-style behavior in real incidents. This module uses only benign, well-known destinations to demonstrate the correlation technique safely.

3. Step-by-Step Procedure

Run one or both of the following safe, controlled commands from the monitored VM:

```
Test-NetConnection example.com -Port 443  
nslookup example.com
```

Both target only the well-known, benign example.com domain reserved by IANA for documentation and testing purposes. Do not substitute an unknown, untrusted, or malicious destination.

4. Investigate

- Originating process (powershell.exe for Test-NetConnection, or the resolver process for nslookup)
- Destination host/domain
- Destination IP address
- Destination port
- Protocol
- Timestamp
- User and hostname context
- The corresponding LimaCharlie network telemetry event

5. Demonstration Flow

```
PowerShell  
|  
v  
Network Connection  
|  
v  
Destination  
|  
v  
Port  
|
```

v
Timestamp

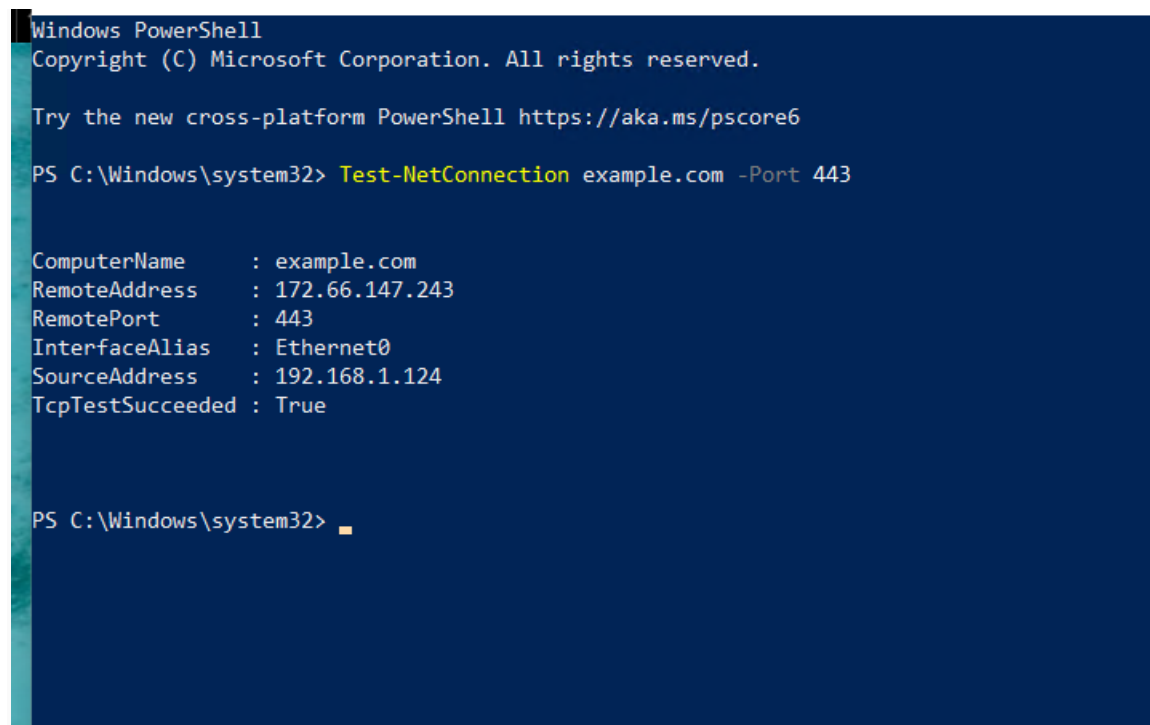
Process-to-network correlation example:

powershell.exe
|
v
Network Connection
|
v
example.com:443

6. Figures

Figure INC4.1 — Network Test Activity Executed

Purpose: Show the controlled Test-NetConnection / nslookup command being run.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> Test-NetConnection example.com -Port 443

ComputerName      : example.com
RemoteAddress     : 172.66.147.243
RemotePort        : 443
InterfaceAlias    : Ethernet0
SourceAddress     : 192.168.1.124
TcpTestSucceeded  : True

PS C:\Windows\system32>
```

Evidence to capture:

- Terminal window
- Command executed
- Command output (connection succeeded / DNS result)

Observation: Not captured in supplied evidence for this figure.

Figure INC4.2 — Detection Rule Triggered (Process-to-Network)

Purpose: Evidence that Detection 4 evaluated and fired on this network event.

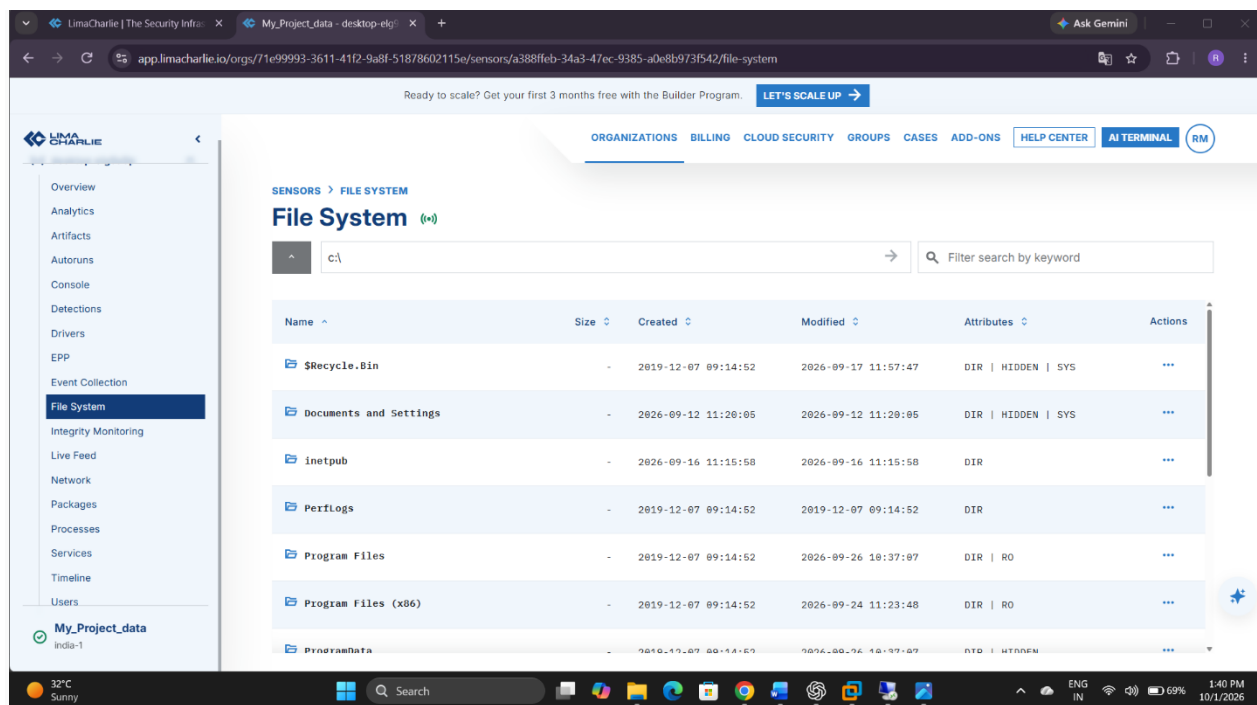
Evidence to capture:

- Originating process
- Destination host/IP
- Destination port
- Protocol

Observation: Not captured in supplied evidence for this figure.

Figure INC4.4 — Process-to-Network Correlation View

Purpose: Evidence linking the specific process to the specific network destination.



Evidence to capture:

- Process name/PID
- Correlated destination and port

Observation: Not captured in supplied evidence for this figure.

7. Important Note on Labeling

This activity targets a benign, well-known documentation domain. Do not label it malicious merely because it generated a network event — the goal of this incident is to demonstrate the correlation mechanism, not to simulate an actual command-and-control channel.

8. MITRE ATT&CK Mapping

Map according to the actual observed behavior. Benign connectivity-test traffic to a known documentation domain does not, by itself, map to a specific adversary technique; note this explicitly in the report rather than forcing a mapping.

9. Incident Evidence Table

Table 16. Incident 4 Evidence

Evidence	Value
Incident ID	Not assigned in supplied evidence
Hostname	Not captured in the connectivity-test screenshot
Username	Not captured in supplied evidence
Timestamp	Not captured in supplied evidence
Process	PowerShell console shown; no correlated process event
Parent Process	Not captured in supplied evidence
Child Process	Not captured in supplied evidence
PID	Not captured in supplied evidence
Command Line	Test-NetConnection example.com -Port 443
Network Connection	example.com, TCP/443; TcpTestSucceeded=True; source address shown as 192.168.1.124
Detection Rule	No process-to-network rule firing is shown
Alert	Not captured in supplied evidence
ATT&CK Technique	Not applicable to this benign connectivity check; no technique assigned
Severity	No severity assigned; benign connectivity check
Response Action	No response action captured
Verification	Connectivity test succeeded; EDR process-to-network correlation was not verified

10. Incident Timeline

Table 17. Incident 4 Timeline

Time	Event	Evidence	Analyst Observation
Not captured	Activity generated	Test-NetConnection example.com -Port 443; TcpTestSucceeded=True	Benign connectivity test succeeded; screenshot shows source 192.168.1.124.
Not captured	Telemetry received	Not captured	No correlated LimaCharlie network event supplied.
Not captured	Detection triggered	Not captured	No process-to-network detection or alert shown.
Not captured	Alert investigated	Not captured	No alert investigation captured.
Not captured	ATT&CK mapped	Not applicable	No adversary technique is inferred from benign test traffic.
Not captured	Response performed	Not applicable	No response action captured.
Not captured	Activity verified stopped	Not captured	The one-shot connectivity test succeeded; process termination was not documented.

11. Security Relevance

Process-to-network correlation is a key technique for identifying data exfiltration or command-and-control channels in real incidents, since it ties a suspicious process directly to where it is communicating.

Module 9 — Threat Investigation Workflow & Evidence Collection

9.1 Standardized Investigation Workflow

Every alert in this project — regardless of which detection fired — is worked through the same six-step process:

Step 1 — Identify

What happened? Read the alert and identify the triggering event and process.

Step 2 — Validate

Is this activity legitimate or suspicious, given what is known about the environment and the user's expected behavior?

Step 3 — Investigate

Check user, process, parent process, child process, command line, network activity, files, and the surrounding timeline.

Step 4 — Map

Map the observed behavior to a MITRE ATT&CK technique where appropriate — never forcing a mapping onto activity that does not support it.

Step 5 — Respond

Take a proportionate, controlled response action.

Step 6 — Document

Record the incident in the standardized evidence table, timeline, and final incident report.

The six-step workflow is the procedure defined for this lab. The supplied evidence does not establish that it was completed for all four planned scenarios: no custom-rule alert investigation is shown.

9.2 Standardized Evidence Collection

For each planned incident, the evidence table provides sixteen fields. Values are completed only where the screenshots support them; unsupported incident-specific details are marked “Not captured in supplied evidence.”

- Incident ID
- Hostname
- Username
- Timestamp
- Process

- Parent Process
- Child Process
- PID
- Command Line
- Network Connection
- Detection Rule
- Alert
- ATT&CK Technique
- Severity
- Response Action
- Verification

Unknown values are recorded as “Not captured in supplied evidence.” This wording distinguishes missing documentation from a negative test result.

9.3 Incident Timeline Standard

Each incident also records a seven-row timeline (Time / Event / Evidence / Analyst Observation) covering: activity generated, telemetry received, detection triggered, alert investigated, ATT&CK mapped, response performed, and activity verified stopped. Timestamps used must be the actual timestamps observed in the lab, not estimated or invented values.

Module 10 — MITRE ATT&CK Mapping

The table below summarizes planned ATT&CK mappings alongside the evidence actually supplied. A behavior mapping does not establish malicious intent or successful detection.

Table 18. MITRE ATT&CK Mapping

Observed Activity	ATT&CK Technique	ID	Evidence
PowerShell execution	Command and Scripting Interpreter: PowerShell	T1059.001	Incident 1 (Module 5)
Windows Command Shell (cmd.exe spawning PowerShell)	Command and Scripting Interpreter: Windows Command Shell	T1059.003	Incident 2 (Module 6)
Scheduled task creation	Scheduled Task/Job: Scheduled Task	T1053.005	Incident 3 (Module 7)
Network activity (benign test connection)	Mapped according to actual observed behavior — no forced mapping	N/A	Incident 4 (Module 8)

10.1 Mapping Rationale

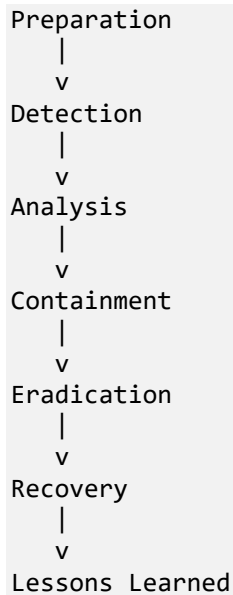
The ATT&CK table describes technique mappings that are appropriate when the corresponding behavior is observed. The screenshots support benign PowerShell use and scheduled-task creation; they do not show a confirmed adversary, a detection alert, or the proposed cmd.exe-to-PowerShell process chain. The benign network test is left unmapped.

10.2 MITRE Screenshot Requirement

A consolidated screenshot of the ATT&CK Navigator (or an equivalent LimaCharlie ATT&CK view, if available) showing all techniques observed in this project should be captured — see the Screenshot Master List, Section 12, IDs LC-MITRE-01 through LC-MITRE-05.

Module 11 — Incident Response

11.1 Incident Response Lifecycle



11.2 Lifecycle Applied to This Project

Detection

A configured detection rule can generate an alert when matching telemetry is received. The supplied screenshots show rule configuration but do not show a rule firing or an alert for the scenarios in this report.

Analysis

The analyst investigates the endpoint using the standardized workflow in Module 9.

Containment

Stop or isolate the suspicious activity when appropriate — e.g., terminate a controlled test process.

Eradication

Remove the lab persistence artifact or other test artifact (e.g., the scheduled task in Incident 3).

Recovery

Verify the endpoint is functioning normally after the response action.

Lessons Learned

Document what happened, why it was detected, what evidence was available, and what could improve detection.

11.3 Per-Incident Response Documentation

For each planned scenario, document the following six fields where evidence is available:

- Detection — How was the activity detected? (Which rule, which alert.)
- Analysis — What evidence confirmed the activity? (Process, command line, ancestry, network.)
- Containment — What controlled action was taken?
- Eradication — What laboratory artifact was removed, if any?
- Recovery — Was the endpoint returned to normal, and how was that confirmed?
- Lessons Learned — What could improve the detection or investigation process?

The supplied evidence confirms creation and later removal of the benign scheduled task LimaCharlieLabTest. It also shows benign connectivity checks. Other response actions, process termination, endpoint recovery checks, and incident sign-off are not captured.

Only safe, controlled response actions were used, consistent with the safety requirements in Section 15:

- Stopping a lab test process when appropriate.
- Removing the benign scheduled task created in Incident 3.
- Terminating controlled test activity once evidence had been captured.
- Verifying the endpoint continued functioning normally after each response action.

This project intentionally excludes: real malware, credential theft, destructive payloads, ransomware, data destruction, unauthorized/real persistence mechanisms, real-world exploitation, and malicious external targets. It is a defensive detection-and-response laboratory only.

11.5 Incident Response Figures

See the response-action figures embedded within each incident module (Figures INC1.8, INC3.5-INC3.6). A consolidated response-actions screenshot set is indexed as LC-IR-01 through LC-IR-07 in Section 12.

False Positives and Detection Tuning

Behavior-based detections inevitably risk flagging legitimate activity. Understanding and documenting these cases is part of responsible detection engineering.

Legitimate Activity That Can Trigger These Detections

- Administrator scripts that use PowerShell for routine system management.
- Software installers that launch PowerShell or cmd.exe as part of setup.
- IT automation tools (patch management, configuration management) that spawn interpreters.
- Scheduled maintenance tasks created by legitimate software.
- PowerShell making legitimate outbound connections (e.g., module downloads, update checks).

Tuning Without Disabling

Rather than simply turning a detection off after a false positive, tuning should narrow the condition — for example, excluding a specific known-benign signed binary path, requiring an additional corroborating signal (unusual parent process, unusual command-line flags, or an unexpected network destination) before alerting, or downgrading severity for well-understood benign patterns while still keeping the underlying visibility intact.

Module 12 — SOC Dashboard and Final Results

12.1 Target / Example Dashboard

This is a target/example dashboard only. It is not an actual-results summary and must not be interpreted as a count of alerts or resolved incidents from the supplied lab evidence.

```
=====
      WINDOWS  EDR  SOC
=====
Endpoints                1
Total Alerts              4
  Low                    0
  Medium                 2
  High                   2
  Critical               0
Incidents                 4
  PowerShell             1
  Process Chain          1
  Persistence            1
  Network Activity       1
MITRE Techniques          3+
Resolved Incidents       4
=====
```

A separate SIEM is not required for this project to be valid — LimaCharlie serves as the central EDR platform, while this dashboard/report is built and maintained separately as a summary artifact.

12.2 Actual Dashboard From Supplied Evidence

Table 19. Dashboard Results from Supplied Evidence

Metric	Actual Value
Total endpoints	1 Windows endpoint shown online (organization list also contains other sensors)
Total alerts	Not verifiable; no custom-rule alert list or complete count supplied
Alerts — Low	Not captured
Alerts — Medium	Not captured
Alerts — High	Not captured
Alerts — Critical	Not captured
Incidents total	No confirmed security incident; two benign activity tests are evidenced (scheduled task and connectivity)
MITRE techniques observed	T1053.005 behavior evidenced; PowerShell use is shown but not as adversary activity
Resolved incidents	Not applicable; incident resolution is not established

12.3 Dashboard Sections Specification

- Total endpoints monitored

- Total alerts generated
- Alerts broken down by severity
- Total incidents and incident status (open/resolved)
- Detection type breakdown (PowerShell / process chain / persistence / network)
- MITRE techniques observed
- Resolved incidents count
- Timeline of all incidents
- Key investigation findings summary

Screenshot Master List

This index records the figure IDs planned for the report. An ID in this index does not by itself mean that the corresponding screenshot was captured; observations in the relevant section state when evidence is absent or mismatched.

Module 1 — Endpoint Setup (LC-01-01 to LC-01-12)

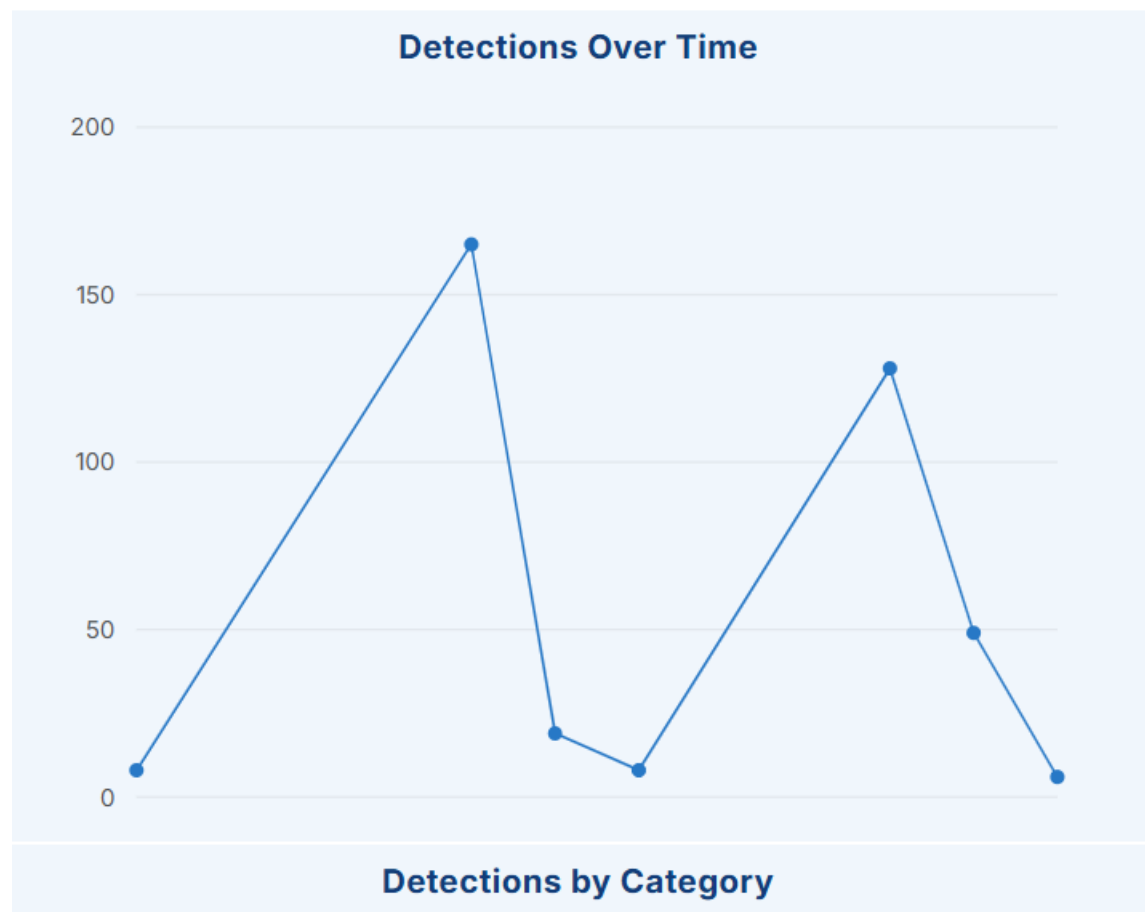


Table 20. Endpoint Setup Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-01-01	Fig. 1.1	Windows VM Powered On and Logged In
LC-01-02	Fig. 1.2	Windows Endpoint Hostname
LC-01-03	Fig. 1.3	Logged-In User Identity
LC-01-04	Fig. 1.4	Windows Version and Architecture
LC-01-05	Fig. 1.5	CPU and RAM Configuration
LC-01-06	Fig. 1.6	Disk / Storage Configuration
LC-01-07	Fig. 1.7	IP Configuration
LC-01-08	Fig. 1.8	Internet Connectivity Test
LC-01-09	Fig. 1.9	PowerShell Version
LC-01-10	Fig. 1.10	Windows Event Viewer Baseline
LC-01-11	Fig. 1.11	Windows Defender / Security Status
LC-01-12	Fig. 1.12	Administrator Privilege Check

Module 2 — LimaCharlie Deployment (LC-02-01 to LC-02-06)

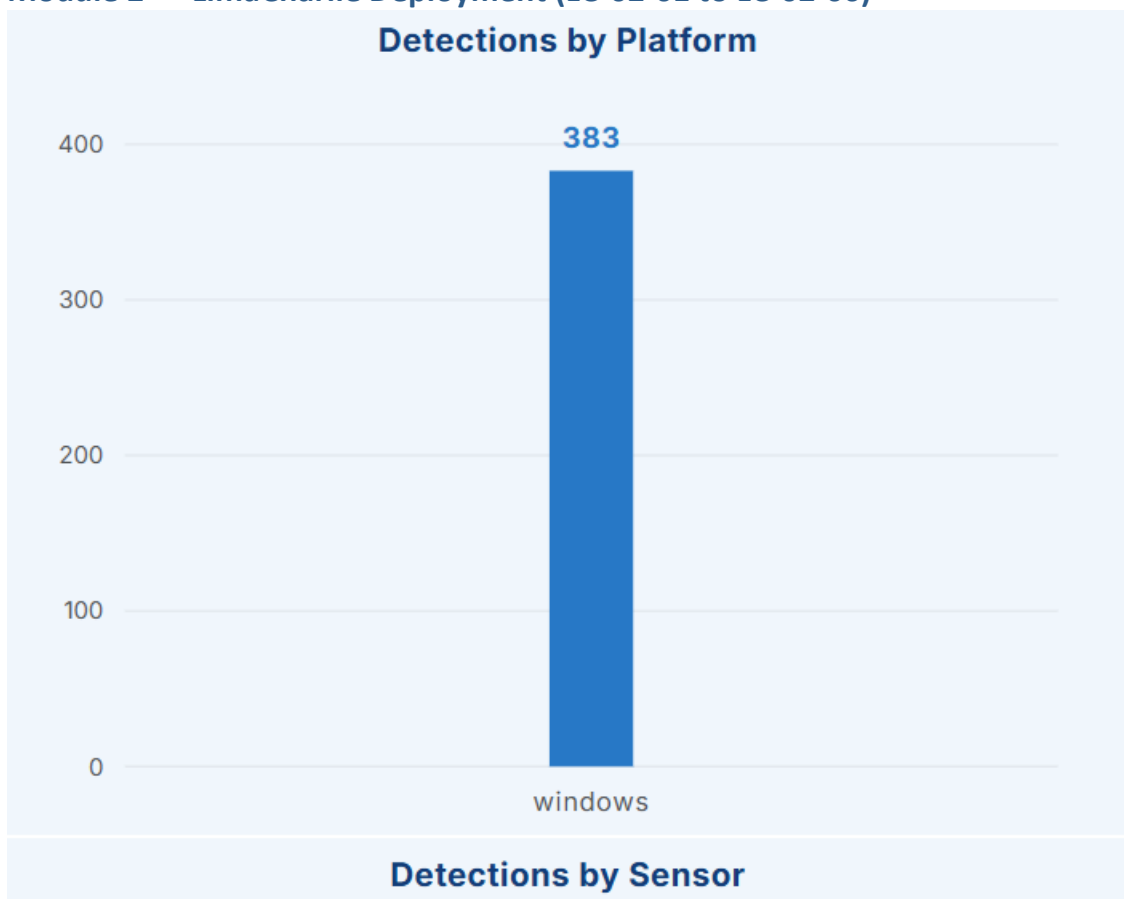


Table 21. LimaCharlie Deployment Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-02-01	Fig. 2.1	LimaCharlie Account / Login Screen
LC-02-02	Fig. 2.2	LimaCharlie Organization Created/Selected
LC-02-03	Fig. 2.3	Sensor Installer / Installation Key Generated
LC-02-04	Fig. 2.4	Sensor Installation in Progress / Complete
LC-02-05	Fig. 2.5	LimaCharlie Sensor Service Status
LC-02-06	Fig. 2.6	Endpoint Visible and Online in LimaCharlie Console

Module 3 — Telemetry (LC-03-01 to LC-03-03)

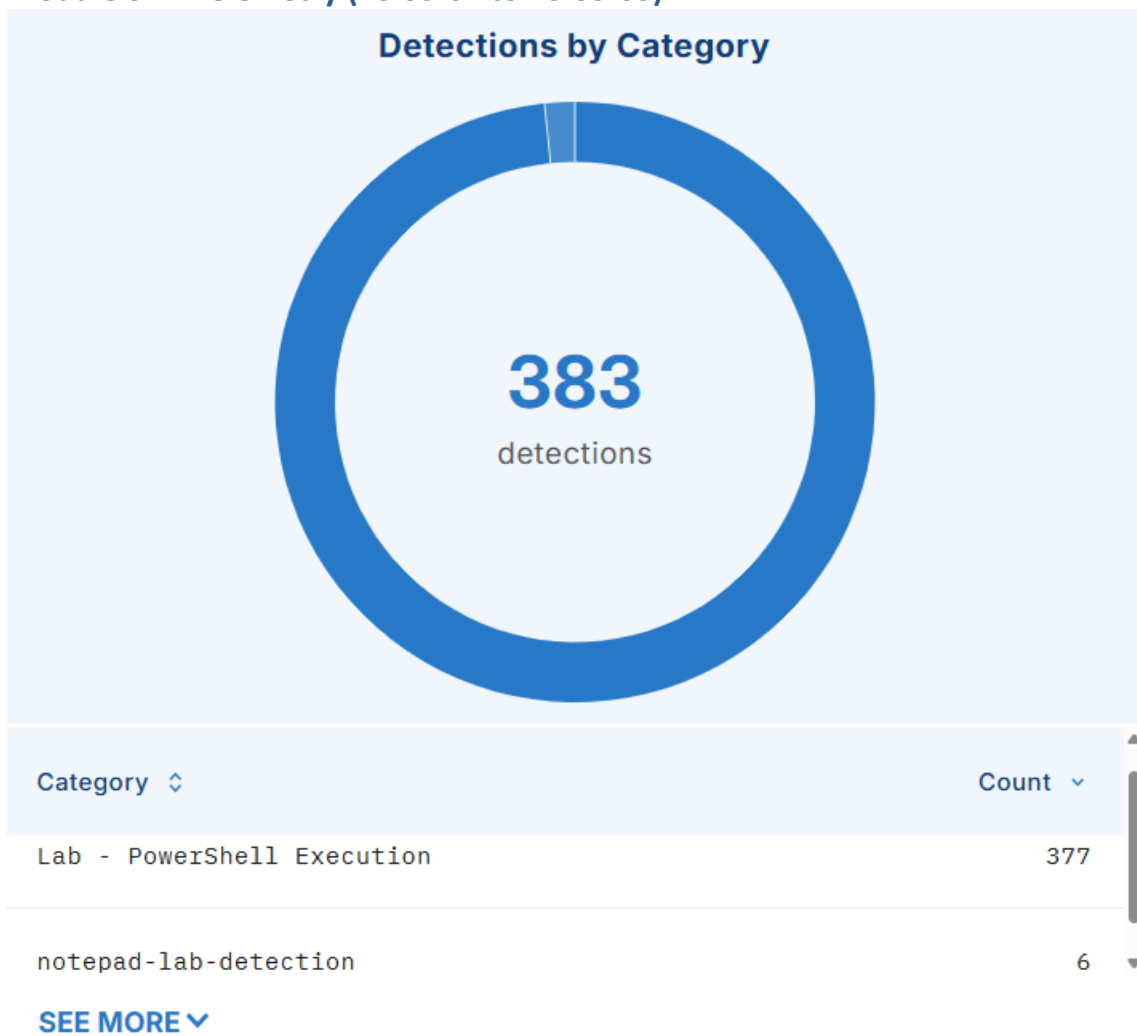


Table 22. Telemetry Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-03-01	Fig. 3.1	LimaCharlie Timeline / Event Stream View
LC-03-02	Fig. 3.2	Process Telemetry Detail

Screenshot ID	Figure Ref.	Title
LC-03-03	Fig. 3.3	Network Telemetry Detail

Detection Engineering (LC-DE-01 to LC-DE-04)

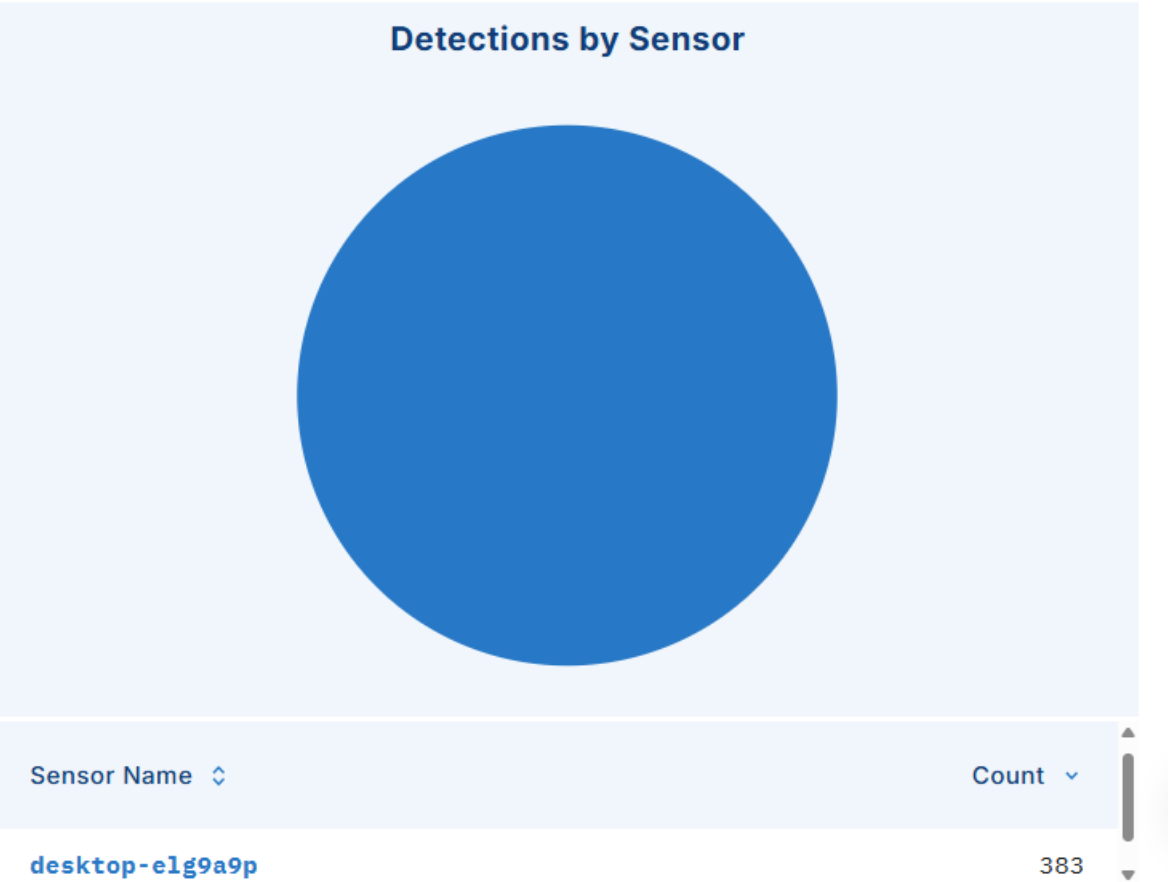


Table 23. Detection Engineering Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-DE-01	Fig. DE.1	Detection 1 Rule Configuration (PowerShell Execution)
LC-DE-02	Fig. DE.2	Detection 2 Rule Configuration (cmd->PowerShell)
LC-DE-03	Fig. DE.3	Detection 3 Rule Configuration (Scheduled Task)
LC-DE-04	Fig. DE.4	Detection 4 Rule Configuration (Process-to-Network)

Incident 1 (LC-INC1-01 to LC-INC1-09)

Table 24. Incident 1 Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-INC1-01	Fig. INC1.1	PowerShell Test Activity Executed
LC-INC1-02	Fig. INC1.2	Detection Rule Triggered
LC-INC1-03	Fig. INC1.3	Alert Generated

Screenshot ID	Figure Ref.	Title
LC-INC1-04	Fig. INC1.4	Process Detail
LC-INC1-05	Fig. INC1.5	Command-Line Detail
LC-INC1-06	Fig. INC1.6	Investigation View
LC-INC1-07	Fig. INC1.7	MITRE ATT&CK Mapping
LC-INC1-08	Fig. INC1.8	Response Action
LC-INC1-09	Fig. INC1.9	Verification

Incident 2 (LC-INC2-01 to LC-INC2-05)

Table 25. Incident 2 Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-INC2-01	Fig. INC2.1	Process Chain Generated
LC-INC2-02	Fig. INC2.2	Detection Rule Triggered (cmd->PowerShell)
LC-INC2-03	Fig. INC2.3	Full Process Tree in LimaCharlie
LC-INC2-04	Fig. INC2.4	Command Lines for Each Process in the Chain
LC-INC2-05	Fig. INC2.5	Investigation Summary

Incident 3 (LC-INC3-01 to LC-INC3-06)

Table 26. Incident 3 Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-INC3-01	Fig. INC3.1	Scheduled Task Creation Command
LC-INC3-02	Fig. INC3.2	Detection Rule Triggered (Scheduled Task)
LC-INC3-03	Fig. INC3.3	Alert and Task Detail in LimaCharlie
LC-INC3-04	Fig. INC3.4	Task Scheduler Confirmation
LC-INC3-05	Fig. INC3.5	Task Removal (Cleanup)
LC-INC3-06	Fig. INC3.6	Post-Cleanup Verification

Incident 4 (LC-INC4-01 to LC-INC4-04)

Table 27. Incident 4 Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-INC4-01	Fig. INC4.1	Network Test Activity Executed
LC-INC4-02	Fig. INC4.2	Detection Rule Triggered (Process-to-Network)
LC-INC4-03	Fig. INC4.3	Network Telemetry Detail
LC-INC4-04	Fig. INC4.4	Process-to-Network Correlation View

MITRE ATT&CK (LC-MITRE-01 to LC-MITRE-05)

Table 28. MITRE ATT&CK Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-MITRE-01	—	Consolidated ATT&CK Table (this report, Module 10)
LC-MITRE-02	—	ATT&CK Navigator / Console View — T1059.001
LC-MITRE-03	—	ATT&CK Navigator / Console View — T1059.003
LC-MITRE-04	—	ATT&CK Navigator / Console View — T1053.005
LC-MITRE-05	—	Full Technique Coverage Summary

Incident Response (LC-IR-01 to LC-IR-07)

Table 29. Incident Response Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-IR-01	—	Detection Summary (all 4 incidents)
LC-IR-02	—	Analysis Notes Summary
LC-IR-03	—	Containment Action — PowerShell
LC-IR-04	—	Eradication Action — Scheduled Task Removal
LC-IR-05	—	Recovery Verification
LC-IR-06	—	Lessons Learned Notes
LC-IR-07	—	Final Response Sign-off

Dashboard (LC-DASH-01 to LC-DASH-06)

Table 30. Dashboard Screenshot Index

Screenshot ID	Figure Ref.	Title
LC-DASH-01	—	Dashboard Overview
LC-DASH-02	—	Alerts by Severity
LC-DASH-03	—	Incidents by Type
LC-DASH-04	—	MITRE Technique Coverage
LC-DASH-05	—	Incident Timeline (all incidents)
LC-DASH-06	—	Resolved Incidents Summary

Screenshot Placeholder Design Standard

Every placeholder in this report follows the design shown below rather than an unlabeled blank space:



Figure: LC-INC1-04

Title: LimaCharlie PowerShell Detection Alert

Purpose: Evidence that the detection rule triggered.

Required: Alert name, endpoint, timestamp, relevant event info.

Observation: Not captured in supplied evidence OBSERVATION HERE]

General Troubleshooting Reference

Module-specific troubleshooting tables appear within Modules 1-3. The two additional cross-cutting scenarios below apply across all four incidents.

Detection Does Not Trigger

Table 31. Detection Troubleshooting

Check	Action
Is the sensor online?	Re-confirm sensor status (Module 2.7) before assuming a rule problem.
Is telemetry arriving?	Re-confirm the telemetry checklist (Module 3.3) for the relevant event type.
Is the detection enabled?	Confirm the rule is saved and toggled on/active in the console, not left in a draft/disabled state.
Does the activity match the detection logic exactly?	Re-check field names/operators (e.g., exact process path, exact parent-process match) against what was actually generated.
Is the time range correct?	Widen the console's alert/time-range filter.
Is the correct endpoint selected?	Confirm the console is viewing the sensor matching the baseline hostname.

Process Tree Missing or Incomplete

Table 32. Process Tree Troubleshooting

Possible Cause	Action
Telemetry gap	Confirm the sensor was online and connected for the entire duration of the test activity, not just at the end.
Process already exited before telemetry captured ancestry	Re-run the test activity and check the tree shortly after execution rather than after the process has fully exited.
Viewing wrong time window	Widen the console's time-range filter to include the full test sequence.
Platform-specific display setting	Consult current LimaCharlie documentation for how process-tree views are configured in the console version in use — do not assume a setting not yet verified.

Chapter 5 — Controlled Attack/Activity Simulation (Summary)

The report describes four planned controlled activity categories in Modules 5–8: PowerShell execution, a process-ancestry chain, scheduled-task persistence, and outbound network activity. The evidence confirms selected benign PowerShell commands, scheduled-task creation and removal, and a benign connectivity test. It does not establish that all four proposed incident scenarios were fully executed.

Chapter 6 — Detection and Investigation (Summary)

Module 4 describes detection logic for four scenarios. The supplied screenshots show rule configuration, but the console reports 0 of 13 rules tested and no matching alert, process tree, incident timeline, or response outcome is captured for the proposed scenarios.

Chapter 7 — Incident Response (Summary)

The response lifecycle provides a framework for the planned scenarios. The screenshots confirm removal of the benign scheduled task, but do not establish a complete lifecycle response for each scenario.

Chapter 8 — Results

8.1 Project Results Table

The table records the actual values supported by the supplied evidence and identifies results that could not be verified.

Table 33. Detection and Incident Results

Incident	Detection	Investigation	ATT&CK	Response	Status
PowerShell	PowerShell command activity is visible	No alert or incident investigation shown	T1059.001 (interpreter behavior only)	No response captured	Not validated as a detection
Process Chain	The proposed cmd.exe → PowerShell chain is not shown	No process tree/alert supplied	T1059.001 / T1059.003 planned; not verified	No response captured	Not demonstrated
Persistence	LimaCharlieLabTest task creation and removal are shown	Task query confirms Ready, then absent	T1053.005 (benign task creation)	Task deleted; query confirmed absent	Cleanup verified; detection not demonstrated
Network	Benign example.com TCP/443 test succeeded	No correlated process/network event or alert shown	Not applicable; no forced mapping	No response captured	Connectivity verified; EDR correlation not verified

8.2 Investigation Results

The endpoint screenshots show a Windows 10 Pro system named DESKTOP-ELG9A9P and a LimaCharlie sensor for that hostname displayed online. The screenshots include PowerShell 5.1 output, Windows Security status, and benign command activity. A D&R rules page shows 0 of 13 rules tested, and no custom-rule alert is shown.

8.3 Response Results

The scheduled task LimaCharlieLabTest was created successfully, queried as Ready, deleted successfully, and then absent from a subsequent query. No containment or response action is evidenced for the other scenarios.

8.4 False Positives Observed

No false-positive test results are documented. The report discusses possible false-positive sources as general tuning guidance only.

8.5 Limitations

- Single-endpoint, single-analyst lab scale, not a production multi-host SOC.
- The command activity shown in the supplied screenshots is benign. The evidence does not establish successful detection of a real adversary or complete validation of all planned scenarios.
- Exact LimaCharlie rule syntax and console workflow must be verified against the live platform at the time of testing.

8.6 Observations

The available evidence supports endpoint setup and limited benign activity testing. It does not support a conclusion that all four detections were validated or that four incidents were resolved. Complete detection testing and capture the corresponding event, alert, investigation, and response evidence before reporting those outcomes.

Skills Demonstrated

- Endpoint Detection and Response (EDR) deployment and operation
- SOC operations fundamentals
- Detection engineering (rule design, testing, tuning)
- Threat investigation (process, command-line, and network analysis)
- Process analysis and process-ancestry reasoning
- Network analysis and process-to-network correlation
- Windows security administration and PowerShell
- MITRE ATT&CK technique mapping
- Incident response lifecycle execution
- Evidence collection and chain-of-custody-style documentation
- Security documentation and report writing
- False-positive analysis and detection tuning
- Continuous security monitoring practices

Module 13 — Final Incident Report Summary

This summary records the scheduled-task activity supported by the supplied screenshots. It does not claim a detection or alert that is not evidenced.

Table 34. Final Incident Summary

Field	Value
Incident ID	Not assigned in supplied evidence
Incident Name	Benign scheduled-task creation and cleanup test
Detection Date/Time	Not captured; the screenshot shows a scheduled next-run time of 1 October 2026, 23:59
Affected Endpoint	Not identifiable from the task-command screenshots
User	Not captured in the task-command screenshots
Severity	No severity assigned; benign controlled lab activity
Detection	No LimaCharlie detection or alert is shown. The task creation was confirmed by the command output and a subsequent query.
Investigation	The task LimaCharlieLabTest was configured to run cmd.exe /c echo LabTest once at 23:59. The supplied query showed status Ready.
Evidence	See the Incident 3 evidence table; no LimaCharlie detection event or alert was captured.
MITRE ATT&CK	T1053.005 — Scheduled Task/Job: Scheduled Task (behavior mapping only; no adversary activity established)
Timeline	See the Incident 3 timeline; action times were not captured.
Response	The task was deleted with schtasks /delete /tn "LimaCharlieLabTest" /f.
Remediation	The benign test task was removed.
Recovery	A subsequent query reported that the task could not be found. Broader endpoint recovery checks are not captured.
Final Status	Test artifact cleanup verified; EDR detection outcome not demonstrated.
Lessons Learned	Capture the correlated LimaCharlie event, rule result, alert details, and exact timestamps during the next validation run. Keep configuration evidence separate from detection evidence.

Chapter 9 — Conclusion

9.1 Achievements

The supplied evidence documents endpoint enrollment, selected telemetry views, rule configuration, benign test commands, and cleanup of a scheduled task. It does not establish that four custom detections were validated or that the full incident-response cycle was completed. The report separates verified activity from planned scenarios and identifies the evidence needed to complete validation.

9.2 Skills Demonstrated

See the consolidated Skills Demonstrated section above.

9.3 Security Relevance

The documented setup and evidence workflow are relevant to junior SOC analysis: establishing endpoint context, reviewing telemetry, validating alerts, mapping behavior, responding proportionately, and recording findings. This evidence set demonstrates setup and selected benign activity, while the custom detection and alert-investigation steps remain unverified.

9.4 Future Improvements

- Extend to multiple endpoints to practice cross-host correlation.
- Add Sysmon as a secondary telemetry source and compare coverage against LimaCharlie's native telemetry.
- Introduce additional MITRE ATT&CK technique categories (e.g., credential access simulation using only safe, non-destructive test tooling).
- Automate evidence collection and report generation using the LimaCharlie API.
- Build a proper external SOC dashboard (e.g., a simple web app) consuming LimaCharlie API data instead of a static/manual dashboard.

References

- LimaCharlie Documentation. “Windows Agent Installation.” <https://docs.limacharlie.io/2-sensors-deployment/endpoint-agent/windows/installation/>
- MITRE ATT&CK Enterprise. “Command and Scripting Interpreter: PowerShell” (T1059.001). <https://attack.mitre.org/techniques/T1059/001/>
- MITRE ATT&CK Enterprise. “Scheduled Task/Job: Scheduled Task” (T1053.005). <https://attack.mitre.org/techniques/T1053/005/>
- Microsoft Learn. PowerShell documentation. <https://learn.microsoft.com/powershell> ; “schtasks” command reference. <https://learn.microsoft.com/windows-server/administration/windows-commands/schtasks>
- NIST SP 800-61 Rev. 3. Incident Response Recommendations and Considerations for Cybersecurity Risk Management. April 2025. <https://csrc.nist.gov/pubs/sp/800/61/r3/final>

Appendix

A. Safety Requirements

Every activity performed in this project is controlled and suitable for a private cybersecurity laboratory. Only benign test activity is used. This project does not introduce:

- Real malware
- Credential theft
- Destructive payloads
- Ransomware
- Data destruction
- Unauthorized/real persistence mechanisms
- Real-world exploitation
- Malicious external targets

The Windows VM used throughout this project is the sole controlled endpoint.

B. Quality Requirements

This report preserves the supplied structure, captions, and screenshot evidence. It records supported endpoint values and documented benign actions, and marks unsupported detection, alert, incident, and response outcomes as not captured. Complete detection validation before presenting the four scenarios as resolved incidents.

C. Do-Not-Fabricate Policy

This report uses the supplied screenshots as its evidence base. Values are included only when legible in those images; unsupported host-specific, alert, process, and timing details are marked as not captured. The screenshots show configuration and benign test activity, but do not establish successful validation of all four detection scenarios.